



FACULTAD DE CIENCIAS DE LA VIDA Y TECNOLOGÍAS

Trabajo Autónomo: Desarrollo y Publicación de Páginas WEB

Nombre:

Winter Manuel Povea Bravo

Docente:

Ing. Patricia Alexandra Quiroz Palma

Asignatura:

Aplicaciones Web I

Curso:

5to “A”

Periodo:

2025-2

Contenido

1. Descripción del negocio o tema elegido.....	5
2. Propósito del sistema web	5
3. Requerimientos funcionales y no funcionales	6
3.1. Requerimientos funcionales.....	6
3.2. Requerimientos no funcionales	8
4. Herramientas y Tecnologías de programación (frameworks, librerías, etc) y almacenamiento de datos utilizado.	9
4.1. Entorno de desarrollo y control de versiones.....	9
4.2. Framework y Tecnologías Frontend.....	10
4.3. Gestión del estado y control de acceso	10
4.4. Visualización de datos	11
4.5. Almacenamiento de datos	11
4.6. Control de versiones y repositorio	11
5. Método de publicación y hosting.....	12
5.1. Proceso de construcción del proyecto	12
5.2. Hosting utilizado	12
5.3. Método de publicación	12
5.4. Acceso y funcionamiento del sistema	13
5.5. Consideraciones técnicas.....	13
6. Código fuente completo.....	13
6.1. Estructura del proyecto	13
6.2. Login	14
6.2.1. LoginView.vue	14
6.2.2. login.css	15
6.2.3. authService.js.....	17
6.2.4. auth.js.....	17
6.3. Dashboard	18

6.3.1.	index.js.....	18
6.3.2.	DashboardView.vue	19
6.3.3.	dashboard.css.....	19
6.3.4.	Sidebar.vue	22
6.3.5.	Statscards.vue	22
6.3.6.	Topbar.vue.....	24
6.4.	Control de Acceso	24
6.4.1.	AccesosView.vue	24
6.4.2.	controlAcceso.css	26
6.5.	Gestion de Usuarios	29
6.5.1.	UsuariosView.vue	29
6.5.2.	Usuarios.css	31
6.6.	Gestion de parqueaderos.....	32
6.6.1.	ParqueaderosView.vue	32
6.6.2.	parqueadero.css.....	34
6.7.	Gestión de Visitantes	36
6.7.1.	VisitantesViews.vue	36
6.7.2.	RegistroVisitantes.css.....	39
		40
6.8.	Archivos .js	41
6.9.	Enlace a Github	43
6.10.	Enlace al Proyecto en Vercel	43
7.	Manual de Usuario del Sistema Web	44
7.1.	Introducción	44
7.2.	Requisitos para el uso del sistema	44
7.3.	Acceso al sistema.....	44
7.4.	Dashboard (panel principal).....	45

7.5. Funcionalidades para el rol guardia	46
7.5.1. Control de Acceso Vehicular.....	46
7.5.2. Registro de Visitantes.....	47
7.6. Funcionalidades para el rol Administrador	48
7.6.1. Gestión de parqueaderos.....	48
7.6.2. Gestión de Usuarios Autorizados	49
7.6.3. Gestión de visitantes y descarga de historial	50
7.7. Cierre de Sesión	50
7.8. Consideraciones finales	50
8. Conclusión del Proyecto.....	51
9. Bibliografía.....	51

Índice de Ilustraciones

Ilustración 1. Estructura del Proyecto en Visual Studio Code	13
Ilustración 2. Código Login View	14
Ilustración 3. Código login.css 1	15
Ilustración 4. Código login.css 2	16
Ilustración 5. Código authService.js.....	17
Ilustración 6. Código auth.js.....	17
Ilustración 7. Código index.js.....	18
Ilustración 8. DashboardView.vue	19
Ilustración 9. Código dashboard.css 1	19
Ilustración 10. Código dashboard.css 2	20
Ilustración 11. Código dashboard.css 3	21
Ilustración 12. Código Statscards.vue 1	22
Ilustración 13. Código Statscards.vue 2	23
Ilustración 14. Código Topbar.vue	24
Ilustración 15. Código AccesosView.vue 1.....	24
Ilustración 16. Código AccesosView.vue 2.....	25
Ilustración 17. Código controlAcceso.css 1	26

Ilustración 18. Código controlAcceso.css 2	27
Ilustración 19. Código controlAcceso.css 3	28
Ilustración 20. Código UsuariosView.vue 1	29
Ilustración 21. Código UsuariosView.vue 2.....	30
Ilustración 22. Código Usuarios.css	31
Ilustración 23. Código ParqueaderosView.vue 1	32
Ilustración 24. Código ParqueaderosView.vue	33
Ilustración 25. Código parqueadero.css 1	34
Ilustración 26. Código parqueadero.css 2.....	35
Ilustración 27. Código VisitantesView.vue	36
Ilustración 28. Código VisitantesView.vue	37
Ilustración 29. Código VisitantesView.vue	38
Ilustración 30. Código RegistroVisitantes.css 1	39
Ilustración 31. Código RegistroVisitantes.css 2	40
Ilustración 32. Código parqueaderosService.js	41
Ilustración 33. Código usuariosService.js	42
Ilustración 34. Código visitantesService.js	43
Ilustración 35. Interfaz Acceso al Sistema	45
Ilustración 36. Interfaz Dashboard	46
Ilustración 37. Interfaz Control de Acceso	47
Ilustración 38. Interfaz Gestión de Visitantes	48
Ilustración 39. Interfaz Gestión de Parqueaderos	49
Ilustración 40. Interfaz Gestión de Usuarios	50

1. Descripción del negocio o tema elegido

El Sistema de Control de Parqueaderos ULEAM surge como una solución tecnológica orientada a optimizar la gestión del acceso vehicular dentro del campus universitario de la Universidad Laica Eloy Alfaro de Manabí.

Actualmente, el control de ingreso y salida de vehículos de estudiantes, docentes, personal administrativo y visitantes se realiza mediante procesos manuales, lo cual genera retrasos, errores de registro y falta de trazabilidad en los movimientos vehiculares.

El sistema propuesto permitirá automatizar el registro, monitoreo y control del flujo vehicular, asignando roles específicos a los diferentes tipos de usuarios (administrador, guardia de seguridad, usuario autorizado y visitante). Además, proporcionará una plataforma web centralizada que facilite el acceso seguro, el registro de eventos, la gestión de parqueaderos y la consulta de historiales, contribuyendo a mejorar la seguridad, eficiencia y orden en las zonas de parqueo institucionales.

2. Propósito del sistema web

El propósito del Sistema Web de Control de Parqueaderos ULEAM es brindar una solución tecnológica que permita automatizar, organizar y centralizar la gestión del acceso vehicular dentro del campus universitario de la Universidad Laica Eloy Alfaro de Manabí, reemplazando los procesos manuales por una plataforma web accesible, estructurada y eficiente.

Este sistema tiene como objetivo principal optimizar el control de ingreso y salida de vehículos, facilitando el registro de usuarios autorizados y visitantes, la administración de parqueaderos disponibles y la visualización de información relevante en tiempo real, contribuyendo así a mejorar la seguridad, el orden y la trazabilidad de los movimientos vehiculares dentro de la institución.

De manera específica, el sistema web busca:

- Reducir errores humanos asociados al registro manual de datos.
- Agilizar los procesos de control vehicular realizados por el personal de seguridad.

- Mantener un historial organizado de accesos, usuarios, visitantes y parqueaderos.
- Proporcionar una interfaz intuitiva y fácil de usar para administradores y guardias.
- Sentar las bases para futuras integraciones con bases de datos y servicios dinámicos.

Al estar desarrollado como una aplicación web y publicada en un servicio de hosting, el sistema permite su acceso desde distintos dispositivos dentro de la red institucional, demostrando la aplicación práctica de los conocimientos adquiridos en la asignatura Aplicaciones Web I y evidenciando el uso de tecnologías modernas para la solución de problemas reales en el entorno universitario.

3. Requerimientos funcionales y no funcionales

3.1. Requerimientos funcionales

Los requerimientos funcionales describen las acciones y comportamientos que el sistema web debe realizar para cumplir con su propósito. A continuación, se detallan los principales requerimientos del Sistema de Control de Parqueaderos ULEAM:

- **RF-01 Autenticación de usuarios**

El sistema debe permitir el inicio de sesión mediante credenciales válidas, diferenciando el acceso según el rol del usuario (administrador o guardia).

- **RF-02 Gestión de roles y permisos**

El sistema debe restringir el acceso a las funcionalidades según el rol del usuario:

- El guardia debe tener acceso al dashboard, control de acceso y registro de visitantes.
- El administrador debe tener acceso al dashboard, gestión de parqueaderos, gestión de usuarios y registro de visitantes.

- **RF-03 Visualización del dashboard**

El sistema debe mostrar un panel principal con información relevante, incluyendo:

- Cantidad de usuarios autorizados dentro del campus.

- Clasificación por tipo de usuario (estudiantes, docentes, personal administrativo y visitantes).
- Gráfico de barras con datos estadísticos.
- Registro resumido de las últimas salidas.

- **RF-04 Registro y control de acceso vehicular**

El sistema debe permitir buscar usuarios autorizados mediante número de placa o cédula, verificando su existencia en los datos almacenados y registrando automáticamente:

- Datos del usuario.
- Tipo de acción (ingreso o salida).
- Fecha y hora del evento.

- **RF-05 Gestión de parqueaderos**

El sistema debe permitir al administrador registrar y visualizar información de los parqueaderos, incluyendo:

- Nombre del parqueadero.
- Ubicación dentro del campus.
- Capacidad total.
- Espacios disponibles.
- Estado (activo o inactivo).

- **RF-06 Gestión de usuarios autorizados**

El sistema debe permitir registrar usuarios autorizados del campus, almacenando los siguientes datos:

- Nombres completos.
- Número de cédula.
- Tipo de usuario.
- Facultad o departamento.
- Número de placa.
- Tipo de vehículo.
- Correo institucional.

Además, debe permitir listar y buscar usuarios registrados mediante un cuadro de búsqueda.

- **RF-07 Validación de datos**

El sistema debe validar los datos ingresados en los formularios, verificando campos obligatorios, formato de cédula, correo institucional y número de placa antes de permitir el registro.

- **RF-08 Registro de visitantes**

El sistema debe permitir registrar visitantes externos, almacenando información personal, destino, motivo de la visita y registrando automáticamente la hora de ingreso y salida.

- **RF-09 Descarga de historial de visitantes**

El sistema debe permitir al administrador descargar el historial de visitas en formato JSON.

- **RF-10 Persistencia de datos local**

El sistema debe conservar la información registrada utilizando almacenamiento local (localStorage o archivos JSON), evitando la pérdida de datos al recargar la página.

3.2. Requerimientos no funcionales

Los requerimientos no funcionales describen las características de calidad, rendimiento y usabilidad del sistema web:

- **RNF-01 Usabilidad**

El sistema debe contar con una interfaz simple, intuitiva y fácil de usar, permitiendo que guardias y administradores interactúen con la aplicación sin necesidad de capacitación avanzada.

- **RNF-02 Diseño visual**

El sistema debe mantener un diseño coherente con la identidad institucional de la ULEAM, utilizando colores representativos y una distribución clara de los elementos visuales.

- **RNF-03 Rendimiento**

El sistema debe responder de manera rápida a las acciones del usuario, reduciendo los tiempos de carga y navegación mediante una arquitectura SPA desarrollada en Vue.

- **RNF-04 Disponibilidad**

El sistema debe estar disponible a través de un servicio de hosting web, permitiendo el acceso desde navegadores modernos sin necesidad de instalación adicional.

- **RNF-05 Compatibilidad**

El sistema debe ser compatible con los principales navegadores web actuales y funcionar correctamente en dispositivos de escritorio.

- **RNF-06 Seguridad básica**

El sistema debe restringir el acceso a las funcionalidades mediante autenticación de usuarios y control de roles, evitando el acceso no autorizado a módulos administrativos.

- **RNF-07 Mantenibilidad**

El código del sistema debe estar estructurado y organizado de forma modular, facilitando futuras mejoras, integraciones con bases de datos y ampliaciones de funcionalidad.

- **RNF-08 Persistencia de estado**

El sistema debe mantener la información y el estado de los datos aun cuando la página sea recargada, evitando reinicios no deseados del sistema.

4. Herramientas y Tecnologías de programación (frameworks, librerías, etc) y almacenamiento de datos utilizado.

El desarrollo del Sistema Web de Control de Parqueaderos ULEAM se realizó utilizando tecnologías modernas orientadas a la construcción de aplicaciones web de tipo SPA (Single Page Application), priorizando la modularidad de carpetas y archivos, el rendimiento y la facilidad de mantenimiento.

4.1. Entorno de desarrollo y control de versiones

El proyecto fue desarrollado utilizando el editor de código Visual Studio Code, el cual facilitó la escritura, organización y depuración del código fuente mediante extensiones especializadas para Vue y JavaScript.

Desde el inicio del desarrollo, el sistema fue vinculado a un repositorio en GitHub, permitiendo el control de versiones a través de commits y pushes realizados directamente desde el entorno de desarrollo. Esto permitió llevar un registro detallado de los cambios realizados durante la evolución del proyecto.

4.2. Framework y Tecnologías Frontend

El sistema fue desarrollado utilizando el framework Vue.js, el cual permitió la creación de una aplicación web reactiva y de una sola página (SPA), evitando recargas completas y mejorando la experiencia del usuario.

Las tecnologías principales utilizadas fueron:

- Vue.js: Framework principal para la construcción de la aplicación.
- HTML5: Definición de la estructura de los componentes y vistas.
- CSS3: Diseño visual y estilos personalizados con identidad institucional.
- JavaScript (ES6+): Lógica de negocio, validaciones y control de eventos.

El proyecto fue desarrollado utilizando Node.js como entorno de ejecución para la gestión de dependencias y herramientas de desarrollo. Mediante npm, se instalaron y administraron las librerías necesarias para el funcionamiento del sistema, las cuales se encuentran definidas en el archivo package.json.

Node.js permitió ejecutar el servidor de desarrollo, compilar el proyecto mediante Vite y generar la versión final del sistema web para su publicación en un servicio de hosting estático.

4.3. Gestión del estado y control de acceso

Para la gestión del estado global de la aplicación y el control de los roles de acceso, se utilizó Pinia, librería oficial de Vue para el manejo de estados.

Pinia permitió:

- Centralizar la información del usuario autenticado.
- Validar y controlar el acceso a las vistas según el rol (administrador o guardia).
- Mantener la sesión activa durante la navegación dentro de la aplicación SPA.

Gracias a esta implementación, el sistema restringe el acceso a determinadas interfaces según los permisos del usuario autenticado, fortaleciendo la seguridad y organización del sistema.

4.4. Visualización de datos

Para la representación gráfica de la información estadística mostrada en el dashboard, se utilizó la librería Chart.js, específicamente para la generación de gráficos de barras.

Chart.js permitió:

- Visualizar de forma clara los datos relacionados con los usuarios autorizados dentro del campus.
- Mejorar la interpretación de la información mediante gráficos dinámicos.
- Integrar visualizaciones interactivas de manera sencilla dentro de los componentes Vue.

4.5. Almacenamiento de datos

Dado que el sistema fue desarrollado como una aplicación web estática y no se implementó una base de datos en esta fase del proyecto, se utilizó LocalStorage como mecanismo de persistencia de datos en el navegador.

Los datos fueron almacenados utilizando archivos JSON, lo que permitió trabajar con información semiestructurada y simular el comportamiento de una base de datos. Este enfoque fue aplicado para:

- Usuarios autorizados.
- Registro de visitantes.
- Gestión de parqueaderos.
- Historial de accesos vehiculares.

El uso de JSON facilitó la lectura, escritura y manipulación de datos, además de evitar la pérdida de información al recargar la página.

4.6. Control de versiones y repositorio

El código fuente completo del sistema se encuentra alojado en un repositorio de GitHub, permitiendo su descarga, revisión y futura ampliación. El uso de control de versiones aseguró una evolución ordenada del proyecto y facilita la integración de nuevas funcionalidades en etapas posteriores.

5. Método de publicación y hosting

El Sistema Web de Control de Parquaderos ULEAM fue publicado utilizando un método de despliegue para aplicaciones web estáticas, acorde a la arquitectura SPA desarrollada con Vue.js.

5.1. Proceso de construcción del proyecto

Una vez finalizada la fase de desarrollo, el sistema fue preparado para su publicación mediante el uso de Vite, herramienta de construcción del proyecto. Para generar la versión final del sistema se ejecutó el siguiente comando: ‘npm run build’

Este proceso permitió compilar el código fuente y generar los archivos estáticos optimizados (HTML, CSS y JavaScript), los cuales conforman la versión final del sistema web.

5.2. Hosting utilizado

El sistema fue desplegado en la plataforma de hosting Vercel, la cual está especializada en el despliegue de aplicaciones web modernas y sitios estáticos.

Vercel fue seleccionada debido a que:

- Es compatible de forma nativa con proyectos Vue y Vite.
- Permite la integración directa con repositorios de GitHub.
- Ofrece despliegue automático a partir del código fuente.
- Garantiza tiempos de carga rápidos y alta disponibilidad.

5.3. Método de publicación

La publicación del sistema se realizó vinculando el repositorio de GitHub del proyecto con la plataforma Vercel. De esta manera, Vercel se encargó automáticamente de:

- Instalar las dependencias del proyecto.
- Ejecutar el proceso de construcción (npm run build).
- Publicar los archivos generados en un entorno de producción.

Gracias a este método, cada actualización realizada en el repositorio puede reflejarse en la versión publicada del sistema, facilitando el mantenimiento y la evolución del proyecto.

5.4. Acceso y funcionamiento del sistema

Una vez desplegado, el sistema web quedó accesible a través de una URL pública proporcionada por Vercel, permitiendo su uso desde cualquier navegador moderno sin necesidad de instalación adicional.

La aplicación mantiene su comportamiento como SPA, permitiendo la navegación entre módulos de manera fluida y sin recargas completas de página, demostrando su correcto funcionamiento en un entorno real de producción.

5.5. Consideraciones técnicas

En este método de publicación, Node.js se utiliza únicamente durante el proceso de desarrollo y construcción del proyecto. En producción, Vercel sirve únicamente los archivos estáticos generados, lo que reduce la complejidad del sistema y mejora su rendimiento.

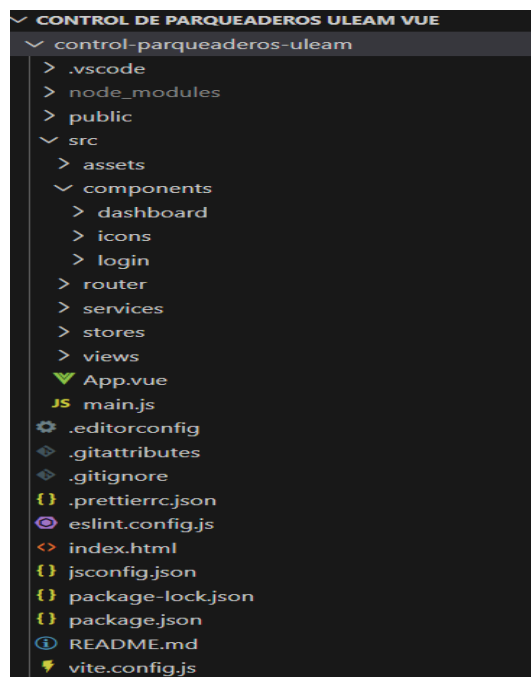
Este enfoque permite que el sistema pueda ser fácilmente escalado o integrado en el futuro con servicios backend y bases de datos, de ser requerido.

6. Código fuente completo

6.1. Estructura del proyecto

Así luce el proyecto acabado, teniendo todo de manera ordenada y modular, para facilitar futuros cambios.

Ilustración 1. Estructura del Proyecto en Visual Studio Code



6.2. Login

6.2.1. LoginView.vue

Ilustración 2. Código Login View

```
1  <template>
2    <div class="login-body">
3      <div class="login-container">
4        <h2>Sistema de Control de Parquaderos</h2>
5        <h3>ULEAM</h3>
6
7        <form @submit.prevent="handleLogin">
8          <div class="form-group">
9            <label>Usuario</label>
10           <input v-model="usuario" type="text" required />
11         </div>
12
13         <div class="form-group">
14           <label>Contraseña</label>
15           <input v-model="password" type="password" required />
16         </div>
17
18         <button class="btn">Iniciar sesión</button>
19
20         <div class="extras">
21           <a href="#">¿Olvidó su contraseña?</a>
22         </div>
23       </form>
24
25       <p v-if="mensaje" :style="{ color: colorMensaje }" class="mensaje">
26         {{ mensaje }}
27       </p>
28     </div>
29   </div>
30 </template>
31
32 <script>
33 import { validarCredenciales } from '@services/authService'
34 import { useAuthStore } from '@stores/auth'
35
36 export default {
37   name: 'LoginForm',
38   data() {
39     return {
40       usuario: '',
41       password: '',
42       mensaje: '',
43       colorMensaje: ''
44     }
45   },
46   methods: {
47     handleLogin() {
48       const { valido, rol } = validarCredenciales(this.usuario, this.password)
49
50       if (valido) {
51         const authStore = useAuthStore()
52         authStore.login(this.usuario, rol)
53         this.mensaje = '✅ Acceso concedido. Redirigiendo...'
54         this.colorMensaje = 'green'
55         setTimeout(() => this.$router.push('/dashboard'), 1500)
56       } else {
57         this.mensaje = '❌ Usuario o contraseña incorrectos'
58         this.colorMensaje = 'red'
59       }
60     }
61   }
62 }
63 </script>
64
65 <style scoped src="@/assets/css/login.css"></style>
```

6.2.2. login.css

Ilustración 3. Código login.css 1

```
1  /* =====
2      Estilo general - Sistema ULEAM
3      ===== */
4
5  .login-body {
6      font-family: 'Poppins', sans-serif;
7      margin: 0;
8      padding: 0;
9      height: 100vh;
10     width: 100vw; /* Agregado para cubrir ancho completo */
11     position: fixed; /* Agregado para posicionar fijo y cubrir toda la pantalla */
12     top: 0;
13     left: 0;
14     display: flex;
15     justify-content: center;
16     align-items: center;
17
18     /* Fondo con imagen institucional difuminada */
19     background:
20         linear-gradient(rgba(0,0,0,0.45), rgba(0,0,0,0.45)),
21         url('/img/ULEAMFOTO.jpg') no-repeat center center fixed;
22     background-size: cover;
23 }
24
25 /* -----
26     Contenedor del formulario
27     ----- */
28 .login-container {
29     background: rgba(255, 255, 255, 0.92);
30     padding: 2rem 3rem;
31     border-radius: 14px;
32     box-shadow: 0 4px 25px rgba(0, 0, 0, 0.2);
33     width: 350px;
34     text-align: center;
35     backdrop-filter: blur(6px);
36 }
37
38 /* -----
39     Títulos
40     ----- */
41 .login-container h2 {
42     color: #2e7d32; /* Verde oscuro institucional */
43     margin-bottom: 0.4rem;
44     font-weight: 600;
45 }
46
47 .login-container h3 {
48     color: #1565c0; /* Azul acento ULEAM */
49     margin-bottom: 2rem;
50     font-weight: 500;
51 }
52
53 /* -----
54     Campos del formulario
55     ----- */
56 .form-group {
57     margin-bottom: 1.2rem;
58     text-align: left;
59 }
60
61 .form-group label {
62     display: block;
63     font-size: 0.9rem;
64     color: #727272;
```

Ilustración 4. Código login.css 2

```
67
68 .form-group input {
69   width: 100%;
70   padding: 0.6rem;
71   border: 1px solid #bbb;
72   border-radius: 8px;
73   font-size: 0.95rem;
74   background-color: #fafafa;
75   transition: border-color 0.3s, box-shadow 0.3s;
76 }
77
78 .form-group input:focus {
79   border-color: #2e7d32;
80   box-shadow: 0 0 6px rgba(46, 125, 50, 0.4);
81   outline: none;
82 }
83
84 /* -----
85   Botón principal
86   ----- */
87 .btn {
88   width: 100%;
89   background-color: #2e7d32; /* Verde principal */
90   color: white;
91   border: none;
92   padding: 0.8rem;
93   border-radius: 8px;
94   cursor: pointer;
95   font-size: 1rem;
96   transition: background 0.3s, transform 0.2s;
97 }
98
99 .btn:hover {
100   background-color: #256628;
101   transform: scale(1.02);
102 }
103
104 /* -----
105   Enlaces y extras
106   ----- */
107 .extras {
108   margin-top: 1rem;
109 }
110
111 .extras a {
112   font-size: 0.9rem;
113   color: #1565c0;
114   text-decoration: none;
115   transition: color 0.3s;
116 }
117
118 .extras a:hover {
119   color: #0d47a1;
120   text-decoration: underline;
121 }
122
123 /* -----
124   Adaptabilidad móvil
125   ----- */
126 @media (max-width: 420px) {
127   .login-container {
128     width: 85%;
129     padding: 2rem;
130   }
131 }
```

6.2.3. authService.js

Ilustración 5. Código authService.js

```
1 export function validarCredenciales(usuario, password) {
2   // Hardcodeado para ejemplo; expande a una base de datos
3   const usuarios = {
4     'Winter Povea': { password: '131705', rol: 'administrador' },
5     'Guardia Juan': { password: '1234', rol: 'guardia' },
6     'Guardia Manuel': { password: '5678', rol: 'guardia' }
7   }
8
9   const userData = usuarios[usuario.trim()]
10  if (userData && password.trim() === userData.password) {
11    return { valido: true, rol: userData.rol }
12  }
13  return { valido: false, rol: null }
14 }
```

6.2.4. auth.js

Ilustración 6. Código auth.js

```
1 import { defineStore } from 'pinia'
2 import { ref } from 'vue'
3
4 export const useAuthStore = defineStore('auth', () => {
5   const user = ref(null) // { nombre: string, rol: 'administrador' | 'guardia' }
6   const isAuthenticated = ref(false)
7
8   // Inicializar desde localStorage
9   function init() {
10     const storedUser = localStorage.getItem('user')
11     const storedAuth = localStorage.getItem('isAuthenticated')
12     if (storedUser && storedAuth === 'true') {
13       user.value = JSON.parse(storedUser)
14       isAuthenticated.value = true
15     }
16   }
17
18   function login(usuario, rol) {
19     user.value = { nombre: usuario, rol }
20     isAuthenticated.value = true
21     // Guardar en localStorage
22     localStorage.setItem('user', JSON.stringify(user.value))
23     localStorage.setItem('isAuthenticated', 'true')
24   }
25
26   function logout() {
27     user.value = null
28     isAuthenticated.value = false
29     // Remover de localStorage
30     localStorage.removeItem('user')
31     localStorage.removeItem('isAuthenticated')
32   }
33
34   return { user, isAuthenticated, login, logout, init }
35 })
```

6.3. Dashboard

6.3.1. index.js

Ilustración 7. Código index.js

```
1 import { createRouter, createWebHistory } from 'vue-router'
2 import LoginView from '@views/LoginView.vue'
3 import { useAuthStore } from '@stores/auth'
4
5 const routes = [
6   {
7     path: '/',
8     name: 'login',
9     component: LoginView
10  },
11  {
12    path: '/dashboard',
13    name: 'dashboard',
14    component: () => import('@views/DashboardView.vue'),
15    meta: { requiresAuth: true, roles: ['administrador', 'guardia'] }
16  },
17  {
18    path: '/accesos',
19    name: 'accesos',
20    component: () => import('@views/AccesosView.vue'),
21    meta: { requiresAuth: true, roles: ['guardia'] }
22  },
23  {
24    path: '/parqueaderos',
25    name: 'parqueaderos',
26    component: () => import('@views/ParqueaderosView.vue'),
27    meta: { requiresAuth: true, roles: ['administrador'] }
28  },
29  {
30    path: '/usuarios',
31    name: 'usuarios',
32    component: () => import('@views/UsuariosView.vue'),
33    meta: { requiresAuth: true, roles: ['administrador'] }
34  },
35  {
36    path: '/visitantes',
37    name: 'visitantes',
38    component: () => import('@views/VisitantesView.vue'),
39    meta: { requiresAuth: true, roles: ['administrador', 'guardia'] }
40  }
41 ]
42
43 const router = createRouter({
44   history: createWebHistory(),
45   routes
46 })
47
48 // Guard de navegación
49 router.beforeEach((to, from, next) => {
50   const authStore = useAuthStore()
51   const requiresAuth = to.meta.requiresAuth
52   const allowedRoles = to.meta.roles
53
54   if (requiresAuth && !authStore.isAuthenticated) {
55     next('/') // Redirige a login si no autenticado
56   } else if (requiresAuth && allowedRoles && !allowedRoles.includes(authStore.user?.rol)) {
57     next('/dashboard') // Redirige a dashboard si no tiene rol permitido
58   } else {
59     next()
60   }
61 })
62
63 export default router
```

6.3.2. DashboardView.vue

Ilustración 8. DashboardView.vue

```
1  <template>
2    <div class="dashboard">
3      <Sidebar />
4
5      <main class="main-content">
6        <Topbar />
7        <StatsCards />
8      </main>
9    </div>
10 </template>
11
12 <script setup>
13 import Sidebar from '@components/dashboard/Sidebar.vue'
14 import Topbar from '@components/dashboard/Topbar.vue'
15 import StatsCards from '@components/dashboard/Statscards.vue'
16 </script>
17
18 <style src="@assets/css/dashboard.css"></style>
19
```

6.3.3. dashboard.css

Ilustración 9. Código dashboard.css 1

```
1  /* =====
2     Estilo general - Dashboard ULEAM
3     ===== */
4
5  .dashboard {
6    width: 100vw;
7    height: 100vh; /* Cambiado de min-height a height para ocupar toda la pantalla */
8    display: flex;
9    font-family: 'Poppins', sans-serif;
10   background-color: #f4f6f8;
11   color: #333;
12   overflow: hidden; /* Evita barras de desplazamiento globales */
13 }
14
15 /* -----
16     Sidebar (menú lateral) - Fijo
17     ----- */
18 .sidebar {
19   background-color: #2e7d32; /* Verde institucional */
20   color: white;
21   width: 213px;
22   height: 100vh; /* Altura completa */
23   position: fixed; /* Fijo a la izquierda */
24   top: 0;
25   left: 0;
26   display: flex;
27   flex-direction: column;
28   justify-content: space-between;
29   padding: 1rem 0;
30   box-shadow: 2px 0 8px rgba(0,0,0,0.1);
31   z-index: 10; /* Asegura que esté encima */
32 }
33
34 .sidebar .logo {
35   text-align: center;
36   margin-bottom: 2rem;
37 }
38
39 .sidebar .logo img {
40   width: 125px;
41   margin-bottom: 0.5rem;
42 }
43
44 .sidebar h2 {
```


Ilustración 10. Código dashboard.css 2

```
45     font-size: 1.1rem;
46     color: #fff;
47 }
48
49 .sidebar nav ul {
50     list-style: none;
51     padding: 0;
52 }
53
54 .sidebar nav ul li {
55     margin: 0.8rem 0;
56 }
57
58 .sidebar nav ul li a {
59     color: #f1f1f1;
60     text-decoration: none;
61     padding: 0.7rem 1.5rem;
62     display: block;
63     transition: background 0.3s, padding-left 0.3s;
64 }
65
66 .sidebar nav ul li a:hover {
67     background-color: #256628;
68     padding-left: 2rem;
69 }
70
71 /* -----
72     Área principal
73     ----- */
74 .main-content {
75     position: fixed;
76     display: flex;
77     flex-direction: column;
78     margin-left: 200px; /* Espacio para el sidebar fijo */
79     padding-top: 70px; /* Espacio para el topbar fijo (ajusta según altura del topbar) */
80     height: calc(100vh - 70px); /* Altura restante */
81     width: calc(100vw - 213px);
82     box-sizing: border-box;
83     overflow-y: auto; /* Solo el contenido se desliza verticalmente */
84 }
85
86 /* Barra superior - Fija */
87 .topbar {
88     background-color: #ffffff;
89     display: flex;
90     justify-content: space-between;
91     align-items: center;
92     padding: 0.8rem 1.5rem;
93     box-shadow: 0 2px 8px rgba(0,0,0,0.1);
94     position: fixed; /* Fijo arriba */
95     top: 0;
96     left: 213px; /* Después del sidebar */
97     width: calc(100vw - 265px); /* Ancho restante */
98     height: 70px; /* Altura fija para calcular padding */
99     z-index: 10;
100 }
101
102 /* Título de la barra superior */
103 .topbar h1 {
104     color: #2e7d32;
105     font-size: 1.4rem;
106 }
107
108 .user-info {
109     display: flex;
110     align-items: center;
111     gap: 1rem;
112 }
113
114 /* Botón de cerrar sesión */
115 .logout-btn {
116     background-color: #ac2f2f;
117     color: white;
118     border: none;
119     padding: 0.5rem 0.9rem;
120     border-radius: 6px;
121     cursor: pointer;
122     transition: background 0.3s;
123 }
```

```

123 }
124
125 .logout-btn:hover {
126   background-color: #8a1c1c;
127 }
128
129 /* -----
130   Tarjetas informativas
131   ----- */
132 .cards {
133   display: grid;
134   grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
135   gap: 1.5rem;
136   padding: 2rem;
137 }
138
139 .card {
140   background-color: #fff;
141   border-radius: 12px;
142   padding: 1rem 1.5rem;
143   box-shadow: 0 2px 8px rgba(0,0,0,0.1);
144   transition: transform 0.2s, box-shadow 0.2s;
145 }
146
147 .card:hover {
148   transform: translateY(-4px);
149   box-shadow: 0 4px 12px rgba(0,0,0,0.15);
150 }
151
152 .card h3 {
153   color: #2e7d32;
154   font-size: 1.1rem;
155   margin-bottom: 0.5rem;
156 }
157
158 /*Color Numeros*/
159 .card p {
160   font-size: 1.6rem;
161   font-weight: bold;
162   color: #333;
163 }
164
165 .card ul {
166   padding-left: 1.2rem;
167   font-size: 0.9rem;
168   color: #555;
169 }
170
171 /* -----
172   Adaptabilidad móvil
173   ----- */
174 @media (max-width: 768px) {
175   .sidebar {
176     width: 70px;
177   }
178
179   .sidebar h2,
180   .sidebar nav ul li a {
181     font-size: 0;
182   }
183
184   .sidebar .logo img {
185     width: 50px;
186   }
187
188   .main-content {
189     margin-left: 70px; /* Ajusta para sidebar móvil */
190     padding-top: 60px; /* Ajusta para topbar móvil */
191     height: calc(100vh - 60px);
192   }
193
194   .topbar {
195     left: 70px;
196     width: calc(100vw - 70px);
197     height: 60px;
198   }
199 }
200
201 /* -----
202   Sección de gráficos
203   ----- */
204 .chart-section {
205   padding: 2rem;
206   background-color: #fff;
207   margin: 2rem;
208   border-radius: 12px;
209   box-shadow: 0 2px 8px rgba(0,0,0,0.1);
210 }
211
212 .chart-section h3 {
213   color: #2e7d32;
214   font-size: 1.3rem;
215   margin-bottom: 1rem;
216   text-align: center;
217 }

```

6.3.4. Sidebar.vue

```
1 <template>
2   <aside class="sidebar">
3     <div class="logo">
4       
5       <h2>Control Parquaderos</h2>
6     </div>
7
8     <nav>
9       <ul>
10        <li><RouterLink to="/dashboard">🏠 Inicio</RouterLink></li>
11        <li v-if="user?.rol === 'guardia'"><RouterLink to="/accesos">🔒 Control de Acceso</RouterLink></li>
12        <li v-if="user?.rol === 'administrador'"><RouterLink to="/parquaderos">🚗 Parquaderos</RouterLink></li>
13        <li v-if="user?.rol === 'administrador'"><RouterLink to="/usuarios">👤 Usuarios</RouterLink></li>
14        <li><RouterLink to="/visitantes">👤 Visitantes</RouterLink></li>
15      </ul>
16    </nav>
17  </aside>
18 </template>
19
20 <script setup>
21 import { RouterLink } from 'vue-router'
22 import { useAuthStore } from '@/stores/auth'
23
24 const { user } = useAuthStore()
25 </script>
```

6.3.5. Statscards.vue

Ilustración 12. Código Statscards.vue 1

```
1 <template>
2   <section class="cards">
3     <div class="card">
4       <h3>Vehículos dentro</h3>
5       <p>{{ vehiculosDentro }}</p>
6     </div>
7
8     <div class="card">
9       <h3>Estudiantes autorizados</h3>
10      <p>{{ estudiantesAutorizados }}</p>
11    </div>
12
13    <div class="card">
14      <h3>Docentes autorizados</h3>
15      <p>{{ docentesAutorizados }}</p>
16    </div>
17
18    <div class="card">
19      <h3>Visitantes activos</h3>
20      <p>{{ visitantesActivos }}</p>
21    </div>
22
23    <div class="card">
24      <h3>Personal administrativo</h3>
25      <p>{{ administrativos }}</p>
26    </div>
27
28    <div class="card">
29      <h3>Últimos ingresos</h3>
30      <ul>
31        <li v-for="ingreso in ultimosIngresos" :key="ingreso.fechaHora">
32          {{ ingreso.fechaHora.split(' ')[1] }} - {{ ingreso.placa }}
33        </li>
34      </ul>
35    </div>
36  </section>
37
38  <section class="chart-section">
39    <h3>Estadísticas Generales</h3>
40    <Bar :data="chartData" :options="chartOptions" />
41  </section>
42 </template>
43
44 <script setup>
45 import { ref, onMounted } from 'vue'
46 import { getUsuariosAutorizados } from '@/services/usuariosService.js'
47 import { getVisitantesActivos } from '@/services/visitantesService.js'
48 import { Bar } from 'vue-chartjs'
49 import {
50   Chart as ChartJS,
51   Title,
52   Tooltip,
53   Legend,
54   BarElement,
55   CategoryScale,
56   LinearScale
57 } from 'chart.js'
58
```

Ilustración 13. Código Statscards.vue 2

```
58
59 ChartJS.register(Title, Tooltip, Legend, BarElement, CategoryScale, LinearScale)
60
61 // Datos reactivos
62 const vehiculosDentro = ref(0)
63 const estudiantesAutorizados = ref(0)
64 const docentesAutorizados = ref(0)
65 const visitantesActivos = ref(0)
66 const administrativos = ref(0)
67 const ultimosIngresos = ref([])
68 const chartData = ref({
69   labels: [],
70   datasets: []
71 })
72 const chartOptions = ref({
73   responsive: true,
74   plugins: {
75     legend: {
76       position: 'top',
77     },
78     title: {
79       display: true,
80       text: 'Estadísticas del Sistema'
81     }
82   }
83 })
84
85 onMounted(() => {
86   // Calcular vehículos dentro
87   const estadoVehiculos = JSON.parse(localStorage.getItem('estadoVehiculos') || '{}')
88   vehiculosDentro.value = Object.values(estadoVehiculos).filter(estado => estado === true).length
89
90   // Calcular usuarios
91   const usuarios = getUsuariosAutorizados()
92   estudiantesAutorizados.value = usuarios.filter(u => u.tipoUsuario === 'Estudiante').length
93   docentesAutorizados.value = usuarios.filter(u => u.tipoUsuario === 'Docente').length
94   administrativos.value = usuarios.filter(u => u.tipoUsuario === 'Administrativo').length
95
96   // Visitantes activos
97   visitantesActivos.value = getVisitantesActivos().length
98
99   // Obtener últimos ingresos
100   const historial = JSON.parse(localStorage.getItem('historialAccesos') || '[]')
101   ultimosIngresos.value = historial
102     .filter(evento => evento.tipo === 'Ingreso')
103     .sort((a, b) => new Date(b.fechaHora) - new Date(a.fechaHora))
104     .slice(0, 3)
105
106   // Datos del gráfico
107   chartData.value = {
108     labels: ['Vehículos Dentro', 'Estudiantes', 'Docentes', 'Administrativos', 'Visitantes'],
109     datasets: [
110       {
111         label: 'Cantidad',
112         data: [
113           vehiculosDentro.value,
114           estudiantesAutorizados.value,
115           docentesAutorizados.value,
116           administrativos.value,
117           visitantesActivos.value
118         ],
119         backgroundColor: [
120           'rgba(255, 99, 132, 0.2)',
121           'rgba(54, 162, 235, 0.2)',
122           'rgba(255, 206, 86, 0.2)',
123           'rgba(75, 192, 192, 0.2)',
124           'rgba(153, 102, 255, 0.2)'
125         ],
126         borderColor: [
127           'rgba(255, 99, 132, 1)',
128           'rgba(54, 162, 235, 1)',
129           'rgba(255, 206, 86, 1)',
130           'rgba(75, 192, 192, 1)',
131           'rgba(153, 102, 255, 1)'
132         ],
133         borderWidth: 1
134       }
135     ]
136   }
137 })
138 </script>
139
```

6.3.6. Topbar.vue

Ilustración 14. Código Topbar.vue

```
1 <template>
2   <header class="topbar">
3     <h1>Panel de Control</h1>
4
5     <div class="user-info">
6       <span>{{ user?.rol === 'administrador' ? 'Administrador' : 'Guardia' }}<b>{{ user?.nombre }}</b></span>
7       <button class="logout-btn" @click="logout">Cerrar sesión</button>
8     </div>
9   </header>
10 </template>
11
12 <script setup>
13 import { useRouter } from 'vue-router'
14 import { useAuthStore } from '@stores/auth'
15
16 const router = useRouter()
17 const { user, logout: storeLogout } = useAuthStore()
18
19 function logout() {
20   storeLogout()
21   router.push('/')
22 }
23 </script>
```

6.4. Control de Acceso

6.4.1. AccesosView.vue

Ilustración 15. Código AccesosView.vue 1

```
1 <template>
2   <div class="dashboard">
3     <Sidebar />
4
5     <main class="main-content">
6       <Topbar />
7
8       <section class="content-section">
9         <h2>Registrar ingreso o salida de vehiculos</h2>
10
11         <form class="control-form" @submit.prevent="registrarEvento" @reset="cancelar">
12           <div class="form-grid">
13             <div class="form-group full-width">
14               <label for="buscar">Buscar por placa o cédula</label>
15               <div class="input-with-btn">
16                 <input type="text" id="buscar" v-model="buscar" placeholder="Ej: ABC-1234 o 1312345678" required>
17                 <button type="button" class="btn-verificar" @click="verificarVehiculo">Verificar</button>
18               </div>
19             </div>
20
21             <div class="form-group">
22               <label for="nombreUsuario">Nombre del usuario</label>
23               <input type="text" id="nombreUsuario" v-model="nombreUsuario" readonly>
24             </div>
25
26             <div class="form-group">
27               <label for="fechaHora">Hora y fecha actual</label>
28               <input type="text" id="fechaHora" v-model="fechaHora" readonly>
29             </div>
30
31             <div class="form-group">
32               <label for="accionDetectada">Acción detectada</label>
33               <input type="text" id="accionDetectada" v-model="accionDetectada" readonly placeholder="Se determinará automáticamente">
34             </div>
35
36             <div class="form-actions">
37               <button type="submit" class="btn-registrar">Registrar evento</button>
38               <button type="reset" class="btn-cancelar">Cancelar</button>
39             </div>
40           </div>
41         </form>
42       </main>
43     </div>
```

Ilustración 16. Código AccesosView.vue 2

```

43 <!-- Tabla de historial -->
44 <h2>Historial reciente de movimientos</h2>
45
46 <table class="tabla historial">
47   <thead>
48     <tr>
49       <th>Placa</th>
50       <th>Usuario</th>
51       <th>Acción</th>
52       <th>Fecha y Hora</th>
53       <th>Acción</th>
54     </tr>
55   </thead>
56   <tbody>
57     <tr v-for="(evento, index) in historial" :key="index">
58       <td>{{ evento.placa }}</td>
59       <td>{{ evento.usuario }}</td>
60       <td><span :class="['accion', evento.tipo.toLowerCase()]>{{ evento.tipo }}</span></td>
61       <td>{{ evento.fechaHora }}</td>
62       <td>
63         <button v-if="evento.tipo === 'Ingreso' && !evento.finalizado" class="btn finalizar-salida" @click="registrarSalidaDirecta(evento, index)">Finalizar</button>
64         <span v-else></span>
65       </td>
66     </tr>
67   </tbody>
68 </table>
69 </section>
70 </div>
71 </template>
72
73 <script setup>
74 const HISTORIAL_KEY = 'historialAccesos'
75 const ESTADO_KEY = 'estadoVehiculos'
76
77 import { ref, reactive, onMounted, watch } from 'vue'
78 import Sidebar from '@components/dashboard/Sidebar.vue'
79 import Topbar from '@components/dashboard/Topbar.vue'
80 import { getUsuariosAutorizados } from '@services/usuariosService.js'
81
82
83
84 // Datos reactivos
85 const buscar = ref('')
86 const nombreUsuario = ref('')
87 const fechaHora = ref('')
88 const accionDetectada = ref('')
89 const historial = ref([])
90 const estado = reactive({}) // placa: true = dentro | false = fuera
91 let encontrado = null
92
93 // Vehículos desde servicio
94 const usuariosService = getUsuariosAutorizados()
95
96 // Guardar en localStorage al cambiar historial o estado
97 onMounted(() => {
98   const historialGuardado = localStorage.getItem(HISTORIAL_KEY)
99   const estadoGuardado = localStorage.getItem(ESTADO_KEY)
100
101   if (historialGuardado) {
102     historial.value = JSON.parse(historialGuardado)
103   }
104
105   if (estadoGuardado) {
106     Object.assign(estado, JSON.parse(estadoGuardado))
107   }
108 })
109
110 // Guardar cambios en historial
111 watch(historial, (nuevoHistorial) => {
112   localStorage.setItem(HISTORIAL_KEY, JSON.stringify(nuevoHistorial))
113 }, { deep: true })
114
115 // Guardar cambios en estado
116 watch(estado, (nuevoEstado) => {
117   localStorage.setItem(ESTADO_KEY, JSON.stringify(nuevoEstado))
118 }, { deep: true })
119
120
121
122 // Función para obtener fecha y hora
123 function obtenerFechaHora() {
124   return new Date().toLocaleString()
125 }
126
127 // Verificar vehículo
128 function verificarVehiculo() {
129   const valor = buscar.value.trim()
130   encontrado = usuariosService.find(u => u.placa.toUpperCase() === valor.toUpperCase() || u.cedula === valor)
131
132   if (encontrado) {
133     nombreUsuario.value = encontrado.nombre
134     fechaHora.value = obtenerFechaHora()
135     const esIngreso = estado[encontrado.placa]
136     accionDetectada.value = esIngreso ? 'Ingreso' : 'Salida'
137   } else {
138     alert('❌ Vehículo o usuario no encontrado')
139     cancelar()
140   }
141 }
142
143 // Registrar evento
144 function registrarEvento() {
145   if (!encontrado) {
146     alert('⚠️ Verifique un vehículo antes de registrar.')
147     return
148   }
149
150   const placa = encontrado.placa
151   const usuario = encontrado.nombre
152   const tipo = accionDetectada.value
153   const momento = fechaHora.value
154
155   // Cambiar estado
156   estado[placa] = tipo === 'Ingreso'
157
158   // Agregar al historial
159   historial.value.unshift({
160     placa,
161     usuario,
162     tipo,
163     fechaHora: momento,
164     finalizado: false
165   })
166
167 // Reset
168 function cancelar() {
169   encontrado = null
170 }
171
172 // Cancelar / Reset
173 function cancelar() {
174   buscar.value = ''
175   nombreUsuario.value = ''
176   fechaHora.value = ''
177   accionDetectada.value = ''
178 }
179
180 // Registrar salida directa desde tabla
181 function registrarSalidaDirecta(evento, index) {
182   if (evento.finalizado) return // seguridad extra
183
184   const momento = obtenerFechaHora()
185   estado[evento.placa] = false
186
187   // Mover el ingreso como finalizado
188   historial.value[index].finalizado = true
189
190   // Registrar la salida como nuevo evento
191   historial.value.unshift({
192     placa: evento.placa,
193     usuario: evento.usuario,
194     tipo: 'Salida',
195     fechaHora: momento
196   })
197
198   alert('🚗 Salida registrada para ${evento.usuario}')
199 }
200
201
202 </script>
203
204 <style src="@assets/css/dashboard.css"></style>
205 <style src="@assets/css/controlAcceso.css"></style>
206

```

6.4.2. controlAcceso.css

Ilustración 17. Código controlAcceso.css 1

```
1  /* =====
2     Control de Acceso Vehicular
3     ===== */
4
5  .content-section {
6     padding: 2rem;
7  }
8
9  /*Colores Títulos*/
10 .content-section h2 {
11     color: #2e7d32;
12     margin-bottom: 1rem;
13 }
14
15 /* Formulario */
16 .control-form {
17     background-color: #fff;
18     padding: 1.5rem;
19     border-radius: 12px;
20     box-shadow: 0 2px 8px rgba(0,0,0,0.1);
21     margin-bottom: 2rem;
22 }
23
24 .form-grid {
25     display: grid;
26     grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
27     gap: 1rem 1.5rem;
28 }
29
30 .form-group {
31     display: flex;
32     flex-direction: column;
33 }
34
35 .form-group.full-width {
36     grid-column: 1 / -1;
37 }
38
39 .form-group label {
40     font-weight: 500;
41     margin-bottom: 0.3rem;
42 }
43
44 .btn-finalizar-salida {
45     background-color: #d32f2f;
46     color: #fff;
47     border: none;
48     padding: 0.5rem 1rem;
49     border-radius: 6px;
50     cursor: pointer;
51     font-size: 0.9rem;
52 }
53 .form-group input,
54 .form-group select {
55     padding: 0.6rem;
56     border: 1px solid #ccc;
57     border-radius: 6px;
58     font-size: 0.95rem;
59 }
```


Ilustración 18. Código controlAcceso.css 2

```
61  /*Color de campos al enfocar*/
62  .form-group input:focus,
63  .form-group select:focus {
64      border-color: #2e7d32;
65      outline: none;
66      box-shadow: 0 0 5px rgba(46,125,50,0.3);
67  }
68
69  /* Campo con botón "Verificar" */
70  .input-with-btn {
71      display: flex;
72      gap: 0.5rem;
73  }
74
75  .input-with-btn input {
76      flex: 1;
77  }
78
79  .btn-verificar {
80      background-color: #1565c0;
81      color: white;
82      border: none;
83      padding: 0.6rem 1rem;
84      border-radius: 6px;
85      cursor: pointer;
86      transition: background 0.3s;
87  }
88
89  .btn-verificar:hover {
90      background-color: #0d47a1;
91  }
92
93  /* Botones principales */
94  .form-actions {
95      display: flex;
96      gap: 1rem;
97      margin-top: 1.5rem;
98  }
99
100  .btn-registrar {
101      background-color: #2e7d32;
102      color: #fff;
103      border: none;
104      padding: 0.7rem 1.5rem;
105      border-radius: 8px;
106      cursor: pointer;
107      transition: background 0.3s;
108  }
109
110  .btn-registrar:hover {
111      background-color: #256628;
112  }
113
114  .btn-cancelar {
115      background-color: #ccc;
116      border: none;
117      padding: 0.7rem 1.5rem;
118      border-radius: 8px;
119      cursor: pointer;
120  }
121
122  .btn-cancelar:hover {
123      background-color: #b3b3b3;
124  }
```



```
1  /* Tabla */
2  .tabla-historial {
3      width: 100%;
4      border-collapse: collapse;
5      background-color: #fff;
6      border-radius: 8px;
7      overflow: hidden;
8      box-shadow: 0 2px 8px rgba(0,0,0,0.1);
9  }
10
11  .tabla-historial th,
12  .tabla-historial td {
13      padding: 0.8rem 1rem;
14      text-align: left;
15      border-bottom: 1px solid #eee;
16  }
17
18  .tabla-historial th {
19      background-color: #2e7d32;
20      color: #fff;
21  }
22
23  .accion {
24      font-weight: bold;
25      padding: 0.2rem 0.5rem;
26      border-radius: 6px;
27      font-size: 0.9rem;
28  }
29
30  .accion.ingreso {
31      background-color: #c8e6c9;
32      color: #1b5e20;
33  }
34
35  .accion.salida {
36      background-color: #ffe0b2;
37      color: #e65100;
38  }
39
40
41  /* Adaptabilidad */
42  @media (max-width: 600px) {
43      .form-actions {
44          flex-direction: column;
45      }
46
47      .input-with-btn {
48          flex-direction: column;
49      }
50
51      .btn-verificar {
52          width: 100%;
53      }
54  }
```

6.5. Gestion de Usuarios

6.5.1. UsuariosView.vue

Ilustración 20. Código UsuariosView.vue 1

```
1 <template>
2   <div class="dashboard">
3     <Sidebar />
4
5     <main class="main-content">
6       <Topbar />
7
8       <section class="content-section">
9         <h2>Registrar persona autorizada</h2>
10
11         <form class="registro-form" @submit.prevent="guardarUsuario" @reset="cancelar">
12           <div class="form-grid">
13             <div class="form-group">
14               <label for="nombre">Nombre completo</label>
15               <input type="text" id="nombre" v-model="form.nombre" required>
16             </div>
17
18             <div class="form-group">
19               <label for="cedula">Cédula / ID ULEAM</label>
20               <input type="text" id="cedula" v-model="form.cedula" required>
21             </div>
22
23             <div class="form-group">
24               <label for="tipoUsuario">Tipo de usuario</label>
25               <select id="tipoUsuario" v-model="form.tipoUsuario" required>
26                 <option value="">Seleccione</option>
27                 <option value="profesor">Profesor</option>
28                 <option value="estudiante">Estudiante</option>
29                 <option value="administrativo">Personal administrativo</option>
30               </select>
31             </div>
32
33             <div class="form-group">
34               <label for="facultad">Facultad / Departamento</label>
35               <input type="text" id="facultad" v-model="form.facultad" required>
36             </div>
37
38             <div class="form-group">
39               <label for="placa">Número de placa</label>
40               <input type="text" id="placa" v-model="form.placa" required>
41             </div>
42
43             <div class="form-group">
44               <label for="tipoVehiculo">Tipo de vehiculo</label>
45               <select id="tipoVehiculo" v-model="form.tipoVehiculo" required>
46                 <option value="">Seleccione</option>
47                 <option value="auto">Auto</option>
48                 <option value="moto">Moto</option>
49               </select>
50             </div>
51
52             <div class="form-group">
53               <label for="correo">Correo institucional</label>
54               <input type="email" id="correo" v-model="form.correo" required>
55             </div>
56
57             <div class="form-actions">
58               <button type="submit" class="btn-registrar">Registrar</button>
59               <button type="reset" class="btn-cancelar">Cancelar</button>
60             </div>
61           </form>
62
63           <!-- Tabla usuarios -->
64           <h2>Listado de personas autorizadas</h2>
65
66           <div class="buscador">
67             <input
68               type="text"
69               id="buscadorUsuarios"
70               v-model="buscador"
71               placeholder="🔍 Buscar por nombre, cédula, placa, facultad, tipo, vehiculo..."
72             >
73           </div>
74
75           <table class="tabla-autorizados">
76             <thead>
77               <tr>
78                 <th>ID</th>
79                 <th>Nombre</th>
80                 <th>Cédula</th>
81                 <th>Tipo</th>
82                 <th>Facultad / Departamento</th>
83                 <th>Placa</th>
84                 <th>Vehiculo</th>
85                 <th>Correo</th>
86                 <th>Acciones</th>
87               </tr>
88             </thead>
89             <tbody>
90               <tr v-for="(usuario, index) in usuariosFiltrados" :key="usuario.cedula">
91                 <td>{{ String(usuario.indexOf(usuario) + 1).padStart(3, '0') }}</td>
92                 <td>{{ usuario.nombre }}</td>
93                 <td>{{ usuario.cedula }}</td>
94                 <td>{{ usuario.tipoUsuario }}</td>
95                 <td>{{ usuario.facultad }}</td>
96                 <td>{{ usuario.placa }}</td>
97                 <td>{{ usuario.tipoVehiculo }}</td>
98                 <td>{{ usuario.correo }}</td>
99                 <td>
100                   <button class="btn-editar" @click="editarUsuario(usuario)">Editar</button>
101                   <button class="btn-eliminar" @click="eliminarUsuario(usuario)">Eliminar</button>
102                 </td>
103               </tr>
104             </tbody>
105           </table>
106         </section>
107       </main>
108     </div>
109 </template>
110
111 <script setup>
112   import { ref, reactive, computed, onMounted } from 'vue'
113   import Sidebar from '@components/dashboard/Sidebar.vue'
114   import Topbar from '@components/dashboard/Topbar.vue'
115   import { getUsuariosAutorizados, saveUsuariosAutorizados } from '@services/usuariosService.js'
116 </script>
```

Ilustración 21. Código UsuariosView.vue 2

```

1  // Datos reactivos
2  const usuarios = ref([])
3  const buscador = ref('')
4  const form = reactive({
5    nombre: '',
6    cedula: '',
7    tipoUsuario: '',
8    facultad: '',
9    placa: '',
10   tipoVehiculo: '',
11   correo: ''
12 })
13 let editUsuario = null
14
15 // Computed para filtrado
16 const usuariosFiltrados = computed(() => {
17   const texto = buscador.value.toLowerCase().trim()
18   if (!texto) return usuarios.value
19   return usuarios.value.filter(u =>
20     u.nombre.toLowerCase().includes(texto) ||
21     u.cedula.toLowerCase().includes(texto) ||
22     u.placa.toLowerCase().includes(texto) ||
23     u.tipoUsuario.toLowerCase().includes(texto) ||
24     u.tipoVehiculo.toLowerCase().includes(texto) ||
25     u.facultad.toLowerCase().includes(texto)
26   )
27 })
28
29 // Cargar datos al montar
30 onMounted(() => {
31   usuarios.value = getUsuariosAutorizados()
32 })
33
34 // Validaciones
35 function validar(data) {
36   if (!/^\d{10}$/.test(data.cedula)) {
37     alert('⚠ La cédula debe tener exactamente 10 dígitos.')
38     return false
39   }
40   if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(data.correo)) {
41     alert('⚠ Ingrese un correo válido.')
42     return false
43   }
44   if (!/^[A-Z]{3}-\d{4}$/.test(data.placa.toUpperCase())) {
45     alert('⚠ La placa debe tener el formato ABC-1234.')
46     return false
47   }
48   return true
49 }
50
51 // Guardar usuario
52 function guardarUsuario() {
53   const data = {
54     nombre: form.nombre.trim(),
55     cedula: form.cedula.trim(),
56     tipoUsuario: form.tipoUsuario,
57     facultad: form.facultad.trim(),
58     placa: form.placa.trim().toUpperCase(),
59     tipoVehiculo: form.tipoVehiculo,
60     correo: form.correo.trim()
61   }
62
63   if (!validar(data)) return
64
65   if (editUsuario) {
66     // Editar
67     const index = usuarios.value.indexOf(editUsuario)
68     usuarios.value[index] = data
69     alert('✅ Usuario actualizado correctamente.')
70     editUsuario = null
71   } else {
72     // Agregar
73     usuarios.value.push(data)
74     alert('✅ Usuario agregado exitosamente.')
75   }
76
77   saveUsuariosAutorizados(usuarios.value)
78   cancelar()
79 }
80
81 // Editar usuario
82 function editarUsuario(usuario) {
83   Object.assign(form, usuario)
84   editUsuario = usuario
85   // Scroll al formulario
86   document.querySelector('.registro-form').scrollIntoView({ behavior: 'smooth' })
87 }
88
89 // Eliminar usuario
90 function eliminarUsuario(usuario) {
91   if (confirm('¿Eliminar a "${usuario.nombre}"?')) {
92     const index = usuarios.value.indexOf(usuario)
93     usuarios.value.splice(index, 1)
94     saveUsuariosAutorizados(usuarios.value)
95   }
96 }
97
98 // Cancelar
99 function cancelar() {
100   Object.assign(form, {
101     nombre: '',
102     cedula: '',
103     tipoUsuario: '',
104     facultad: '',
105     placa: '',
106     tipoVehiculo: '',
107     correo: ''
108   })
109   editUsuario = null
110 }
111 </script>
112
113 <style src="@/assets/css/dashboard.css"></style>
114 <style src="@/assets/css/Usuarios.css"></style>

```

6.5.2. Usuarios.css

Ilustración 22. Código Usuarios.css

```
1  /* ===== */
2  Gestión de Usuarios autorizados
3  /* ===== */
4
5  .content-section {
6    padding: 2rem;
7  }
8
9  .content-section h2 {
10   color: #2e7d32;
11   margin-bottom: 1rem;
12 }
13
14 /* formulario */
15 .registro-form {
16   background-color: #fff;
17   padding: 1.5rem;
18   border-radius: 12px;
19   box-shadow: 0 2px 8px rgba(0,0,0,0.1);
20   margin-bottom: 2rem;
21 }
22
23 .form-grid {
24   display: grid;
25   grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
26   gap: 1rem 1.5rem;
27 }
28
29 .form-group {
30   display: flex;
31   flex-direction: column;
32 }
33
34 .form-group label {
35   font-weight: 500;
36   margin-bottom: 0.3rem;
37 }
38
39 .form-group input,
40 .form-group select {
41   padding: 0.6rem;
42   border: 1px solid #ccc;
43   border-radius: 6px;
44   font-size: 0.95rem;
45 }
46
47 /*Color de campos al enfocar*/
48 .form-group input:focus,
49 .form-group select:focus {
50   border-color: #2e7d32;
51   outline: none;
52   box-shadow: 0 0 5px rgba(46,125,50,0.3);
53 }
54
55 .form-actions {
56   display: flex;
57   gap: 1rem;
58   margin-top: 1.5rem;
59 }
60
61 /* Botones */
62 .btn-registrar {
63   background-color: #2e7d32;
64   color: #fff;
65   border: none;
66   padding: 0.7rem 1.5rem;
67   border-radius: 8px;
68   cursor: pointer;
69   transition: background 0.3s;
70 }
71
72 .btn-registrar:hover {
73   background-color: #256628;
74 }
75
76 .btn-cancelar {
77   background-color: #ccc;
78   border: none;
79   padding: 0.7rem 1.5rem;
80   border-radius: 8px;
81   cursor: pointer;
82 }
83
84 .btn-cancelar:hover {
85   background-color: #b3b3b3;
86 }
87
88 .buscador input {
89   width: 100%;
90   padding: 10px;
91   margin: 15px 0;
92   font-size: 16px;
93   border-radius: 6px;
94   border: 1px solid #aaa;
95 }
96
97 /* Tabla */
98 .tabla-autorizados {
99   width: 100%;
100   border-collapse: collapse;
101   background-color: #fff;
102   border-radius: 8px;
103   overflow: hidden;
104   box-shadow: 0 2px 8px rgba(0,0,0,0.1);
105 }
106
107 .tabla-autorizados th,
108 .tabla-autorizados td {
109   padding: 0.8rem 1rem;
110   text-align: left;
111   border-bottom: 1px solid #eee;
112 }
113
114 /* Color tabla encabezado */
115 .tabla-autorizados th {
116   background-color: #2e7d32;
117   color: #fff;
118 }
119
120 .btn-editar, .btn-eliminar {
121   border: none;
122   padding: 0.4rem 0.8rem;
123   border-radius: 6px;
124   cursor: pointer;
125   color: white;
126   font-size: 0.85rem;
127   transition: background 0.3s;
128 }
129
130 .btn-editar {
131   background-color: #1565c0;
132 }
133
134 .btn-editar:hover {
135   background-color: #0d47a1;
136 }
137
138 .btn-eliminar {
139   background-color: #ac2f2f;
140 }
141
142 .btn-eliminar:hover {
143   background-color: #8a1c1c;
144 }
```

6.6. Gestion de parqueaderos

6.6.1. ParqueaderosView.vue

Ilustración 23. Código ParqueaderosView.vue 1

```
1 <template>
2   <div class="dashboard">
3     <Sidebar />
4
5     <main class="main-content">
6       <Topbar />
7
8       <section class="content-section">
9         <h2>Registrar nuevo parqueadero</h2>
10
11         <form class="parqueadero-form" @submit.prevent="guardarParqueadero" @reset="cancelar">
12           <div class="form-grid">
13             <div class="form-group">
14               <label for="nombreParqueadero">Nombre del parqueadero</label>
15               <input type="text" id="nombreParqueadero" v-model="form.nombre" placeholder="Ej: Parqueadero Facultad de Comunicación" required>
16             </div>
17
18             <div class="form-group">
19               <label for="ubicacion">Ubicación / Zona</label>
20               <input type="text" id="ubicacion" v-model="form.ubicacion" placeholder="Ej: Sector Norte - Puerta 2" required>
21             </div>
22
23             <div class="form-group">
24               <label for="capacidad">Capacidad total</label>
25               <input type="number" id="capacidad" v-model.number="form.capacidad" min="1" required>
26             </div>
27
28             <div class="form-group">
29               <label for="disponibles">Espacios disponibles</label>
30               <input type="number" id="disponibles" v-model.number="form.disponibles" min="0" required>
31             </div>
32
33             <div class="form-group">
34               <label for="estado">Estado</label>
35               <select id="estado" v-model="form.estado" required>
36                 <option value="">Seleccione</option>
37                 <option value="activo">Activo</option>
38                 <option value="inactivo">Inactivo</option>
39               </select>
40             </div>
41           </div>
42
43           <div class="form-actions">
44             <button type="submit" class="btn-guardar">Guardar</button>
45             <button type="reset" class="btn-cancelar">Cancelar</button>
46           </div>
47         </form>
48
49         <!-- Tabla de parqueaderos -->
50         <h2>Listado de parqueaderos registrados</h2>
51
52         <table class="tabla-parqueaderos">
53           <thead>
54             <tr>
55               <th>ID</th>
56               <th>Nombre</th>
57               <th>Ubicación</th>
58               <th>Capacidad</th>
59               <th>Disponibles</th>
60               <th>Estado</th>
61               <th>Acciones</th>
62             </tr>
63           </thead>
64           <tbody>
65             <tr v-for="(parqueadero, index) in parqueaderos" :key="index">
66               <td>{{ String(index + 1).padStart(3, '0') }}</td>
67               <td>{{ parqueadero.nombre }}</td>
68               <td>{{ parqueadero.ubicacion }}</td>
69               <td>{{ parqueadero.capacidad }}</td>
70               <td>{{ parqueadero.disponibles }}</td>
71               <td><span :class="['estado', parqueadero.estado]">
72                 {{ parqueadero.estado.charAt(0).toUpperCase() + parqueadero.estado.slice(1) }}
73               </span></td>
74               <td>
75                 <button class="btn-editar" @click="editarParqueadero(index)">Editar</button>
76                 <button class="btn-eliminar" @click="eliminarParqueadero(index)">Eliminar</button>
77               </td>
78             </tr>
79           </tbody>
80         </table>
81       </section>
82     </main>
83   </div>
84 </template>
85
```

Ilustración 24. Código ParqueaderosView.vue

```
1 <script setup>
2 import { ref, reactive, onMounted } from 'vue'
3 import Sidebar from '@components/dashboard/Sidebar.vue'
4 import Topbar from '@components/dashboard/Topbar.vue'
5 import { getParqueaderos, saveParqueaderos } from '@services/parqueaderosService.js'
6
7 // Datos reactivos
8 const parqueaderos = ref([])
9 const form = reactive({
10   nombre: '',
11   ubicacion: '',
12   capacidad: null,
13   disponibles: null,
14   estado: ''
15 })
16 let editIndex = null
17
18 // Cargar datos al montar
19 onMounted(() => {
20   parqueaderos.value = getParqueaderos()
21 })
22
23 // Guardar parqueadero
24 function guardarParqueadero() {
25   const data = { ...form }
26
27   if (editIndex !== null) {
28     // Editar
29     parqueaderos.value[editIndex] = data
30     alert('✅ Parqueadero actualizado')
31     editIndex = null
32   } else {
33     // Agregar nuevo
34     parqueaderos.value.push(data)
35     alert('📌 Parqueadero agregado')
36   }
37
38   saveParqueaderos(parqueaderos.value)
39   cancelar()
40 }
41
42 // Editar parqueadero
43 function editarParqueadero(index) {
44   const p = parqueaderos.value[index]
45   Object.assign(form, p)
46   editIndex = index
47   // Scroll al formulario (opcional)
48   document.querySelector('.parqueadero-form').scrollIntoView({ behavior: 'smooth' })
49 }
50
51 // Eliminar parqueadero
52 function eliminarParqueadero(index) {
53   if (confirm(`¿Eliminar "${parqueaderos.value[index].nombre}"?`)) {
54     parqueaderos.value.splice(index, 1)
55     saveParqueaderos(parqueaderos.value)
56   }
57 }
58
59 // Cancelar / Reset
60 function cancelar() {
61   Object.assign(form, {
62     nombre: '',
63     ubicacion: '',
64     capacidad: null,
65     disponibles: null,
66     estado: ''
67   })
68   editIndex = null
69 }
70 </script>
71
72 <style src="@/assets/css/dashboard.css"></style>
73 <style src="@/assets/css/parqueadero.css"></style>
```

6.6.2. parqueadero.css

Ilustración 25. Código *parqueadero.css* 1

```
1  /* =====
2    Gestión de parqueaderos
3    ===== */
4
5  .content-section {
6    padding: 2rem;
7  }
8  /*Colores Títulos*/
9  .content-section h2 {
10   color: #2e7d32;
11   margin-bottom: 1rem;
12 }
13
14 /* Formulario */
15 .parqueadero-form {
16   background-color: #fff;
17   padding: 1.5rem;
18   border-radius: 12px;
19   box-shadow: 0 2px 8px rgba(0,0,0,0.1);
20   margin-bottom: 2rem;
21 }
22
23 .form-grid {
24   display: grid;
25   grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
26   gap: 1rem 1.5rem;
27 }
28
29 .form-group {
30   display: flex;
31   flex-direction: column;
32 }
33
34 .form-group label {
35   font-weight: 500;
36   margin-bottom: 0.3rem;
37 }
38
39 /*Color de campos sin enfocar*/
40 .form-group input,
41 .form-group select {
42   padding: 0.6rem;
43   border: 1px solid #ccc;
44   border-radius: 6px;
45   font-size: 0.95rem;
46 }
47
48 /*Color de campos al enfocar*/
49 .form-group input:focus,
50 .form-group select:focus {
51   border-color: #2e7d32;
52   outline: none;
53   box-shadow: 0 0 5px rgba(46,125,50,0.3);
54 }
55
56 .form-actions {
57   display: flex;
58   gap: 1rem;
59   margin-top: 1.5rem;
60 }
61
62 .btn-guardar {
63   background-color: #2e7d32;
64   color: #fff;
65   border: none;
66   padding: 0.7rem 1.5rem;
67   border-radius: 8px;
68   cursor: pointer;
69   transition: background 0.3s;
70 }
71
72 .btn-guardar:hover {
73   background-color: #256628;
74 }
75
```


Ilustración 26. Código parqueadero.css 2

```
1  .btn-cancelar {
2    background-color: #ccc;
3    border: none;
4    padding: 0.7rem 1.5rem;
5    border-radius: 8px;
6    cursor: pointer;
7  }
8
9  .btn-cancelar:hover {
10   background-color: #b3b3b3;
11 }
12
13 /* Tabla */
14 .tabla-parqueaderos {
15   width: 100%;
16   border-collapse: collapse;
17   background-color: #fff;
18   border-radius: 8px;
19   overflow: hidden;
20   box-shadow: 0 2px 8px rgba(0,0,0,0.1);
21 }
22
23 .tabla-parqueaderos th,
24 .tabla-parqueaderos td {
25   padding: 0.8rem 1rem;
26   text-align: left;
27   border-bottom: 1px solid #eee;
28 }
29
30 .tabla-parqueaderos th {
31   background-color: #2e7d32;
32   color: #fff;
33 }
34
35 .estado {
36   font-weight: bold;
37   padding: 0.2rem 0.5rem;
38   border-radius: 6px;
39   font-size: 0.9rem;
40 }
41
42 .estado.activo {
43   background-color: #c8e6c9;
44   color: #1b5e20;
45 }
46
47 .estado.inactivo {
48   background-color: #ffcdd2;
49   color: #b71c1c;
50 }
51
52 .btn-editar,
53 .btn-eliminar {
54   border: none;
55   padding: 0.4rem 0.8rem;
56   border-radius: 6px;
57   cursor: pointer;
58   color: white;
59   font-size: 0.85rem;
60   transition: background 0.3s;
61 }
62
63 .btn-editar {
64   background-color: #1565c0;
65 }
66
67 .btn-editar:hover {
68   background-color: #0d47a1;
69 }
70
71 .btn-eliminar {
72   background-color: #ac2f2f;
73 }
74
75 .btn-eliminar:hover {
76   background-color: #8a1c1c;
77 }
```


6.7. Gestión de Visitantes

6.7.1. VisitantesViews.vue

Ilustración 27. Código VisitantesView.vue

```
1 <template>
2   <div class="dashboard">
3     <Sidebar />
4
5     <main class="main-content">
6       <Topbar />
7
8       <section class="content-section">
9         <h2>Registro temporal de visitantes</h2>
10
11         <form class="visitante-form" @submit.prevent="registrarVisitante">
12           <div class="form-grid">
13             <div class="form-group">
14               <label for="nombre">Nombre del visitante</label>
15               <input type="text" id="nombre" v-model="form.nombre" placeholder="Ej: José Ramirez" required>
16             </div>
17
18             <div class="form-group">
19               <label for="cedula">Número de cédula</label>
20               <input type="text" id="cedula" v-model="form.cedula" placeholder="Ej: 1312345678" required>
21             </div>
22
23             <div class="form-group">
24               <label for="placa">Placa del vehículo</label>
25               <input type="text" id="placa" v-model="form.placa" placeholder="Ej: ABC-1234">
26             </div>
27
28             <div class="form-group">
29               <label for="destino">Persona / Facultad destino</label>
30               <input type="text" id="destino" v-model="form.destino" placeholder="Ej: Facultad de Ingeniería">
31             </div>
32
33             <div class="form-group full-width">
34               <label for="motivo">Motivo de la visita</label>
35               <input type="text" id="motivo" v-model="form.motivo" placeholder="Ej: Entrega de documentos">
36             </div>
37
38             <div class="form-group">
39               <label for="horaIngreso">Hora de ingreso</label>
40               <input type="text" id="horaIngreso" v-model="form.horaIngreso" readonly>
41             </div>
42           </div>
43
44           <div class="form-actions">
45             <button type="submit" class="btn-registrar">Registrar ingreso</button>
46             <button type="button" class="btn-finalizar" @click="finalizarTodas">Finalizar visita</button>
47           </div>
48         </form>
49
50         <div class="historial-actions" v-if="esAdministrador">
51           <button type="button" class="btn-historial" @click="descargarHistorial">
52             📄 Descargar historial JSON
53           </button>
54         </div>
55
56         <h2>Visitantes activos</h2>
57
58         <table class="tabla-visitantes">
59           <thead>
60             <tr>
61               <th>Nombre</th>
62               <th>Cédula</th>
63               <th>Placa</th>
64               <th>Destino</th>
65               <th>Motivo</th>
66               <th>Hora de ingreso</th>
67               <th>Acción</th>
68             </tr>
69           </thead>
70           <tbody>
71             <tr v-for="(visitante, index) in visitantesActivos" :key="index">
72               <td>{{ visitante.nombre }}</td>
73               <td>{{ visitante.cedula }}</td>
74               <td>{{ visitante.placa || "-" }}</td>
75               <td>{{ visitante.destino }}</td>
76               <td>{{ visitante.motivo }}</td>
77               <td>{{ visitante.horaIngreso }}</td>
78               <td><button class="btn-salida" @click="finalizarVisita(index)">Finalizar</button></td>
79             </tr>
80           </tbody>
81         </table>
82       </section>
83     </main>
84   </div>
85 </template>
```

Ilustración 28. Código VisitantesView.vue

```
1 <script setup>
2 import { ref, reactive, onMounted, computed } from 'vue'
3 import { useAuthStore } from '@/stores/auth'
4 import Sidebar from '@/components/dashboard/Sidebar.vue'
5 import Topbar from '@/components/dashboard/Topbar.vue'
6 import { getVisitantesActivos, saveVisitantesActivos, getHistorialVisitantes, saveHistorialVisitantes } from '@/services/visitantesService.js'
7 // 🏠 Auth / Roles
8 const authStore = useAuthStore()
9
10 const esAdministrador = computed(() => {
11   return authStore.user?.rol === 'administrador'
12 })
13
14
15 // Datos reactivos
16 const visitantesActivos = ref([])
17 const historial = ref([])
18 const form = reactive({
19   nombre: '',
20   cedula: '',
21   placa: '',
22   destino: '',
23   motivo: ''
24 })
25 const horaIngreso = ref('')
26
27 // Cargar datos al montar
28 onMounted(() => {
29   visitantesActivos.value = getVisitantesActivos()
30   historial.value = getHistorialVisitantes()
31   actualizarHoraIngreso()
32 })
33
34 // Actualizar hora de ingreso
35 function actualizarHoraIngreso() {
36   const now = new Date()
37   horaIngreso.value = now.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' })
38 }
39
40 // Validaciones
41 function validarCampos(cedula, placa) {
42   if (!/^\d{10}$/.test(cedula)) {
43     alert('La cédula debe tener 10 dígitos')
44     return false
45   }
46   if (placa && !/^[A-Z]{3}-?\d{4}$/.test(placa.toUpperCase())) {
47     alert('Placa inválida (ABC-1234)')
48     return false
49   }
50   return true
51 }
52
53 // Registrar visitante
54 function registrarVisitante() {
55   const nuevo = {
56     nombre: form.nombre.trim(),
57     cedula: form.cedula.trim(),
58     placa: form.placa.trim().toUpperCase(),
59     destino: form.destino.trim(),
60     motivo: form.motivo.trim(),
61     horaIngreso: horaIngreso.value
62   }
63
64   if (!validarCampos(nuevo.cedula, nuevo.placa)) return
65
66   if (visitantesActivos.value.some(v => v.cedula === nuevo.cedula)) {
67     alert('Este visitante ya está activo')
68     return
69   }
70
71   visitantesActivos.value.push(nuevo)
72   saveVisitantesActivos(visitantesActivos.value)
73
74   // Reset form
75   Object.assign(form, {
76     nombre: '',
77     cedula: '',
78     placa: '',
79     destino: '',
80     motivo: ''
81   })
82   actualizarHoraIngreso()
83 }
84
```

Ilustración 29. Código VisitantesView.vue

```
1 // Finalizar visita individual
2 function finalizarVisita(index) {
3   const v = visitantesActivos.value[index]
4   if (!confirm('¿Finalizar la visita de ${v.nombre}?')) return
5
6   const horaSalida = new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' })
7   const registro = { ...v, horaSalida }
8
9   historial.value.push(registro)
10  saveHistorialVisitantes(historial.value)
11
12  visitantesActivos.value.splice(index, 1)
13  saveVisitantesActivos(visitantesActivos.value)
14
15  alert(`Visita finalizada a las ${horaSalida}`)
16 }
17
18 // Finalizar todas las visitas
19 function finalizarTodas() {
20   if (!visitantesActivos.value.length) {
21     alert('No hay visitantes')
22     return
23   }
24
25   if (!confirm('¿Finalizar todas las visitas?')) return
26
27   const horaSalida = new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' })
28
29   visitantesActivos.value.forEach(v => {
30     const registro = { ...v, horaSalida }
31     historial.value.push(registro)
32   })
33
34   saveHistorialVisitantes(historial.value)
35
36   visitantesActivos.value = []
37   saveVisitantesActivos(visitantesActivos.value)
38 }
39
40 // Descargar historial
41 function descargarHistorial() {
42   if (!esAdministrador.value) {
43     alert('Acceso denegado: solo administradores')
44     return
45   }
46
47   const blob = new Blob(
48     [JSON.stringify(historial.value, null, 2)],
49     { type: 'application/json' }
50   )
51
52   const a = document.createElement('a')
53   a.href = URL.createObjectURL(blob)
54   a.download = 'historialVisitantes.json'
55   a.click()
56 }
57
58 </script>
59
60 <style src="@/assets/css/dashboard.css"></style>
61 <style src="@/assets/css/RegistroVisitantes.css"></style>
```

6.7.2. RegistroVisitantes.css

Ilustración 30. Código RegistroVisitantes.css 1

```
1  /* =====
2     Registro de Visitantes
3     ===== */
4
5  .content-section {
6     padding: 2rem;
7  }
8  /*Colores Títulos*/
9  .content-section h2 {
10     color: #2e7d32;
11     margin-bottom: 1rem;
12  }
13
14  /* Formulario */
15  .visitante-form {
16     background-color: #fff;
17     padding: 1.5rem;
18     border-radius: 12px;
19     box-shadow: 0 2px 8px rgba(0,0,0,0.1);
20     margin-bottom: 2rem;
21  }
22
23  .form-grid {
24     display: grid;
25     grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
26     gap: 1rem 1.5rem;
27  }
28
29  .form-group {
30     display: flex;
31     flex-direction: column;
32  }
33
34  .form-group.full-width {
35     grid-column: 1 / -1;
36  }
37
38  .form-group label {
39     font-weight: 500;
40     margin-bottom: 0.3rem;
41  }
42
43  /*Color de campos sin enfocar*/
44  .form-group input,
45  .form-group select {
46     padding: 0.6rem;
47     border: 1px solid #ccc;
48     border-radius: 6px;
49     font-size: 0.95rem;
50  }
51
52  /*Color de campos al enfocar*/
53  .form-group input:focus,
54  .form-group select:focus {
55     border-color: #2e7d32;
56     outline: none;
57     box-shadow: 0 0 5px rgba(46,125,50,0.3);
58  }
59
60  .form-actions {
61     display: flex;
62     gap: 1rem;
63     margin-top: 1.5rem;
64  }
65
66  .btn-registrar {
67     background-color: #2e7d32;
68     color: #fff;
69     border: none;
70     padding: 0.7rem 1.5rem;
71     border-radius: 8px;
72     cursor: pointer;
73     transition: background 0.3s;
74  }
75
```

```
1  .btn-registrar: hover {
2      background-color: #256628;
3  }
4
5  .btn-finalizar {
6      background-color: #d32f2f;
7      color: #fff;
8      border: none;
9      padding: 0.7rem 1.5rem;
10     border-radius: 8px;
11     cursor: pointer;
12     transition: background 0.3s;
13 }
14
15 .btn-finalizar: hover {
16     background-color: #b71c1c;
17 }
18
19 .historial-actions {
20     margin-bottom: 1rem;
21 }
22
23 .btn-historial {
24     background-color: #1565c0;
25     color: #fff;
26     border: none;
27     padding: 0.7rem 1.5rem;
28     border-radius: 8px;
29     cursor: pointer;
30     transition: background 0.3s;
31 }
32
33 .btn-historial: hover {
34     background-color: #0d47a1;
35 }
36
37 /* Tabla */
38 .tabla-visitantes {
39     width: 100%;
40     border-collapse: collapse;
41     background-color: #fff;
42     border-radius: 8px;
43     overflow: hidden;
44     box-shadow: 0 2px 8px rgba(0,0,0,0.1);
45 }
46
47 .tabla-visitantes th,
48 .tabla-visitantes td {
49     padding: 0.8rem 1rem;
50     text-align: left;
51     border-bottom: 1px solid #eee;
52 }
53
54 .tabla-visitantes th {
55     background-color: #2e7d32;
56     color: #fff;
57 }
58
59 .btn-salida {
60     background-color: #ac2f2f;
61     color: white;
62     border: none;
63     padding: 0.4rem 0.8rem;
64     border-radius: 6px;
65     cursor: pointer;
66     font-size: 0.85rem;
67     transition: background 0.3s;
68 }
69
70 .btn-salida: hover {
71     background-color: #8a1c1c;
72 }
```


6.8. Archivos .js

Ilustración 32. Código `parqueaderosService.js`

```
1 // Servicio para manejar datos de parqueaderos
2 // Carga inicial desde JSON y persiste en localStorage
3
4 const PARQUEADEROS_KEY = 'parqueaderos';
5
6 const initData = [
7   {
8     nombre: "Parqueadero Facultad de Comunicación",
9     ubicacion: "Sector Norte - Puerta 2",
10    capacidad: 40,
11    disponibles: 15,
12    estado: "activo"
13  },
14  {
15    nombre: "Parqueadero Ciencias de la Salud",
16    ubicacion: "Sector Central - Hospital Universitario",
17    capacidad: 60,
18    disponibles: 40,
19    estado: "activo"
20  },
21  {
22    nombre: "Parqueadero Facultad de Ingeniería",
23    ubicacion: "Sector Oeste - Edificio de Ingeniería",
24    capacidad: 80,
25    disponibles: 55,
26    estado: "activo"
27  },
28  {
29    nombre: "Parqueadero Administración Central",
30    ubicacion: "Sector Administrativo - Rectorado",
31    capacidad: 30,
32    disponibles: 10,
33    estado: "activo"
34  },
35  {
36    nombre: "Parqueadero Biblioteca Central",
37    ubicacion: "Sector Académico - Biblioteca",
38    capacidad: 25,
39    disponibles: 5,
40    estado: "activo"
41  },
42  {
43    nombre: "Parqueadero Docentes",
44    ubicacion: "Sector Interno - Acceso Restringido",
45    capacidad: 50,
46    disponibles: 20,
47    estado: "activo"
48  },
49  {
50    nombre: "Parqueadero Visitantes",
51    ubicacion: "Ingreso Principal - Puerta 1",
52    capacidad: 35,
53    disponibles: 12,
54    estado: "activo"
55  }
56 ];
57
58
59
60 export function getParqueaderos() {
61   let parqueaderos = JSON.parse(localStorage.getItem(PARQUEADEROS_KEY)) || [];
62
63   // Agregar parqueaderos nuevos del initData si no existen
64   initData.forEach(init => {
65     const existe = parqueaderos.some(p => p.nombre === init.nombre);
66     if (!existe) {
67       parqueaderos.push(init);
68     }
69   });
70
71   saveParqueaderos(parqueaderos);
72   return parqueaderos;
73 }
74
75
76 export function saveParqueaderos(parqueaderos) {
77   localStorage.setItem(PARQUEADEROS_KEY, JSON.stringify(parqueaderos));
78 }
```

Ilustración 33. Código usuariosService.js

```
1 // Servicio para manejar datos de usuarios autorizados
2
3 const USUARIOS_AUTORIZADOS_KEY = 'usuariosAutorizados';
4
5 const initialData = [
6   {
7     nombre: "Lucía Torres",
8     cedula: "1312345678",
9     tipoUsuario: "Docente",
10    facultad: "Ciencias Administrativas",
11    placa: "ABC-1234",
12    tipoVehiculo: "Automóvil",
13    correo: "l.torres@uleam.edu.ec"
14  },
15  {
16    nombre: "Carlos Mendoza",
17    cedula: "1318765432",
18    tipoUsuario: "Estudiante",
19    facultad: "Ingeniería",
20    placa: "MBD-0987",
21    tipoVehiculo: "Motocicleta",
22    correo: "c.mendoza@uleam.edu.ec"
23  },
24  {
25    nombre: "María Fernanda Ponce",
26    cedula: "1311122233",
27    tipoUsuario: "Administrativo",
28    facultad: "Rectorado",
29    placa: "GHI-4567",
30    tipoVehiculo: "Automóvil",
31    correo: "m.ponce@uleam.edu.ec"
32  },
33  {
34    nombre: "José Luis Zambrano",
35    cedula: "1314455667",
36    tipoUsuario: "Docente",
37    facultad: "Ciencias de la Salud",
38    placa: "JKL-7788",
39    tipoVehiculo: "Automóvil",
40    correo: "j.zambrano@uleam.edu.ec"
41  },
42  {
43    nombre: "Andrea Belén Macías",
44    cedula: "1319988776",
45    tipoUsuario: "Estudiante",
46    facultad: "Comunicación",
47    placa: "PQR-3344",
48    tipoVehiculo: "Motocicleta",
49    correo: "a.macias@uleam.edu.ec"
50  },
51  {
52    nombre: "Luis Alberto Cedeño",
53    cedula: "1312233445",
54    tipoUsuario: "Docente",
55    facultad: "Ingeniería",
56    placa: "STU-8899",
57    tipoVehiculo: "Camioneta",
58    correo: "l.cedeno@uleam.edu.ec"
59  },
60  {
61    nombre: "Diana Carolina Vélez",
62    cedula: "1315566778",
63    tipoUsuario: "Administrativo",
64    facultad: "Biblioteca Central",
65    placa: "VWX-1122",
66    tipoVehiculo: "Automóvil",
67    correo: "d.velez@uleam.edu.ec"
68  },
69  {
70    nombre: "Kevin Andrés Lóor",
71    cedula: "1316677889",
72    tipoUsuario: "Estudiante",
73    facultad: "Educación",
74    placa: "YZA-5566",
75    tipoVehiculo: "Motocicleta",
76    correo: "k.loor@uleam.edu.ec"
77  }
78 ];
79
80 export function getUsuariosAutorizados() {
81   let usuarios = JSON.parse(localStorage.getItem(USUARIOS_AUTORIZADOS_KEY));
82
83   if (!usuarios) {
84     usuarios = initialData;
85     saveUsuariosAutorizados(usuarios);
86   } else {
87     // Agrega nuevos sin borrar los existentes
88     initialData.forEach(nuevo => {
89       if (!usuarios.some(u => u.cedula === nuevo.cedula)) {
90         usuarios.push(nuevo);
91       }
92     });
93     saveUsuariosAutorizados(usuarios);
94   }
95   return usuarios;
96 }
97
98 export function saveUsuariosAutorizados(usuarios) {
99   localStorage.setItem(USUARIOS_AUTORIZADOS_KEY, JSON.stringify(usuarios));
100 }
101
102
103 }
```

Ilustración 34. Código visitantesService.js

```
1 // Servicio para manejar datos de visitantes
2 // Visitantes activos y historial
3
4 const VISITANTES_ACTIVOS_KEY = 'visitantesActivos';
5 const HISTORIAL_VISITANTES_KEY = 'historialVisitantes';
6
7 const initialActivos = []; // Puedes agregar datos iniciales si quieres
8
9 export function getVisitantesActivos() {
10   let visitantes = JSON.parse(localStorage.getItem(VISITANTES_ACTIVOS_KEY));
11   if (!visitantes) {
12     visitantes = initialActivos;
13     saveVisitantesActivos(visitantes);
14   }
15   return visitantes;
16 }
17
18 export function saveVisitantesActivos(visitantes) {
19   localStorage.setItem(VISITANTES_ACTIVOS_KEY, JSON.stringify(visitantes));
20 }
21
22 export function getHistorialVisitantes() {
23   return JSON.parse(localStorage.getItem(HISTORIAL_VISITANTES_KEY)) || [];
24 }
25
26 export function saveHistorialVisitantes(historial) {
27   localStorage.setItem(HISTORIAL_VISITANTES_KEY, JSON.stringify(historial));
28 }
```

6.9. Enlace a Github

<https://github.com/WinterPovea/Aplicaciones-Web-I/tree/main/Control%20de%20Parqueaderos%20ULEAM%20VUE/control-parqueaderos-uleam>

6.10. Enlace al Proyecto en Vercel

<https://control-acceso-uleam.vercel.app/>

7. Manual de Usuario del Sistema Web

7.1. Introducción

El presente manual tiene como objetivo guiar al usuario en el uso del Sistema Web de Control de Parquaderos ULEAM, describiendo de manera clara y sencilla las funcionalidades disponibles según el rol asignado.

El sistema está diseñado para ser utilizado por administradores y guardias de seguridad, permitiendo el control del acceso vehicular, la gestión de usuarios y parquaderos, y el registro de visitantes dentro del campus universitario.

7.2. Requisitos para el uso del sistema

Para utilizar el sistema web se requiere:

- Un navegador web moderno (Google Chrome, Mozilla Firefox o Microsoft Edge).
- Conexión a Internet.
- Credenciales de acceso válidas proporcionadas por el administrador del sistema.

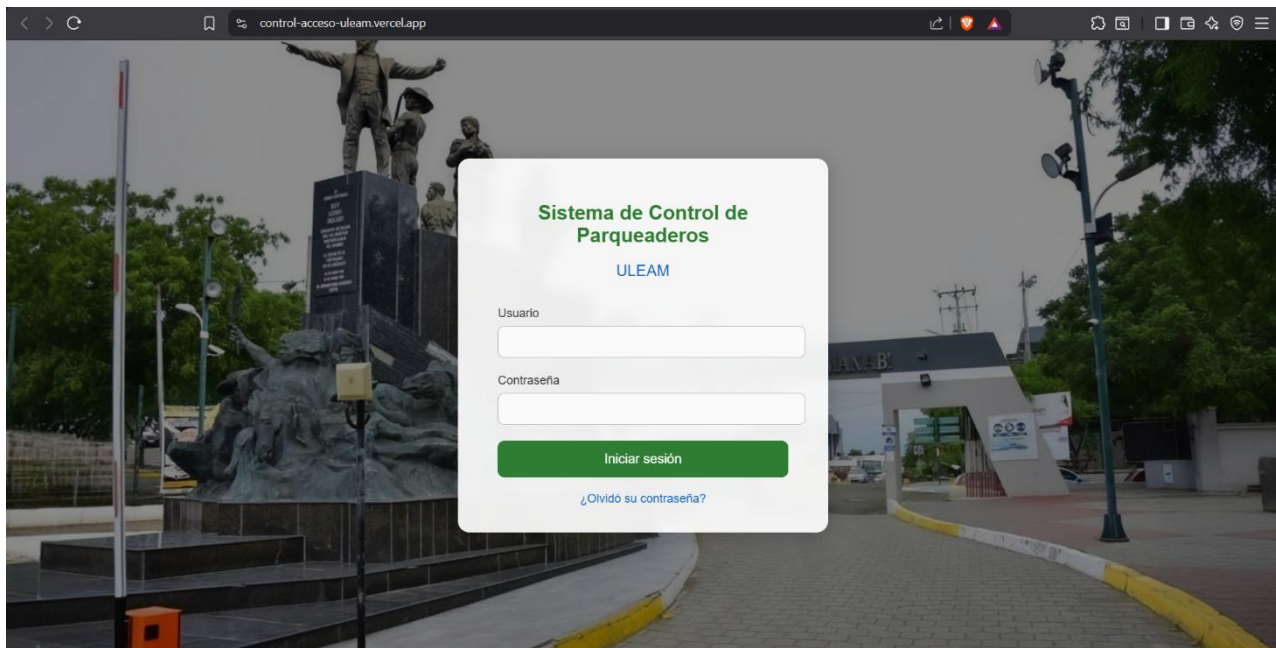
No es necesario instalar ningún software adicional, ya que el sistema funciona completamente desde el navegador.

7.3. Acceso al sistema

1. Ingresar a la URL del sistema web publicada en Vercel.
2. En la pantalla de inicio, introducir el usuario y contraseña.
3. Presionar el botón Iniciar sesión.
4. Si las credenciales son correctas, el sistema redirige automáticamente al Dashboard.

En caso de ingresar datos incorrectos, el sistema mostrará un mensaje de error.

Ilustración 35. Interfaz Acceso al Sistema



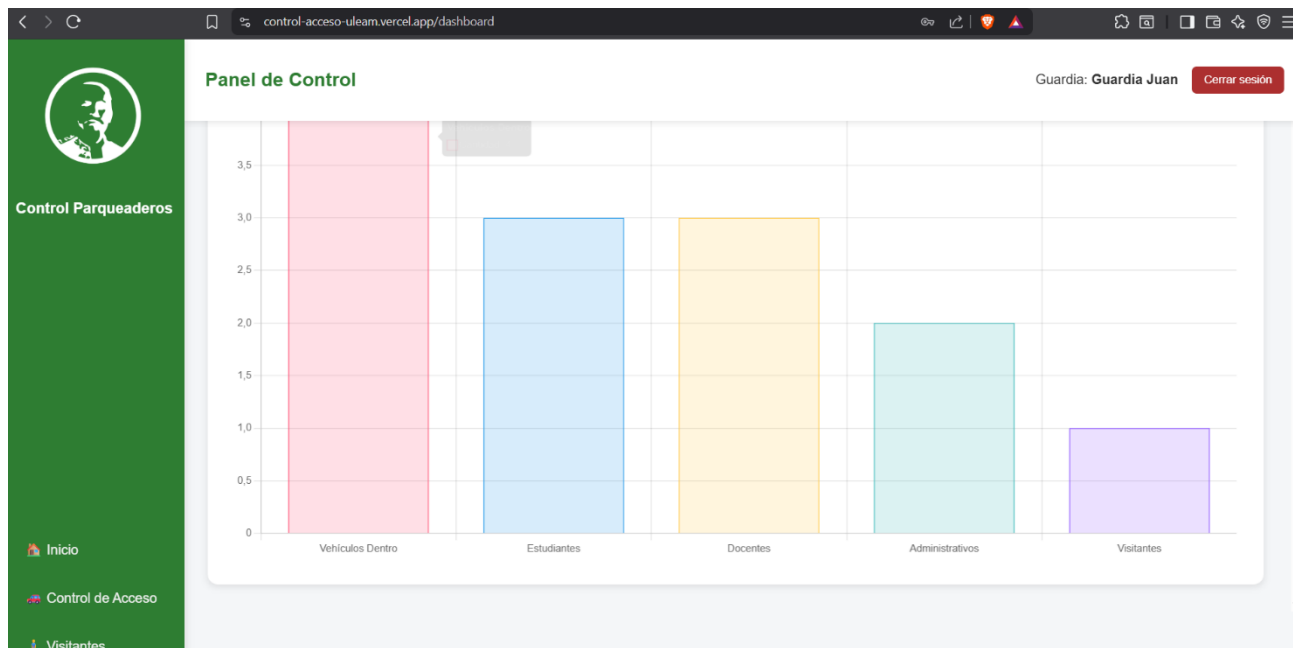
7.4. Dashboard (panel principal)

El dashboard es la pantalla principal del sistema y está disponible para ambos roles.

En esta interfaz se muestra:

- Cantidad de usuarios autorizados dentro del campus.
- Clasificación de usuarios (estudiantes, docentes, personal administrativo y visitantes).
- Gráfico de barras con información estadística.
- Registro resumido de las últimas salidas.

Desde el menú lateral se puede acceder a las demás funcionalidades según el rol del usuario.



7.5. Funcionalidades para el rol guardia

7.5.1. Control de Acceso Vehicular

El guardia puede registrar el ingreso y salida de vehículos autorizados siguiendo estos pasos:

1. Acceder a la opción Control de Acceso desde el menú.
2. Ingresar el número de placa o cédula del usuario.
3. El sistema verifica automáticamente si el usuario está registrado.
4. Si el usuario existe, se muestran sus datos en pantalla.
5. El sistema registra automáticamente:
 - Tipo de acción (ingreso o salida).
 - Fecha y hora del evento.

El registro se almacena y se visualiza en una tabla dentro de la misma interfaz.

Ilustración 37. Interfaz Control de Acceso

Panel de Control Guardia: **Guardia Juan** [Cerrar sesión](#)

Registrar ingreso o salida de vehículos

Buscar por placa o cédula
ABC-1234 [Verificar](#)

Nombre del usuario: Lucía Torres Hora y fecha actual: 11/1/2026, 8:47:31 p. m. Acción detectada: Salida

[Registrar evento](#) [Cancelar](#)

Historial reciente de movimientos

Placa	Usuario	Acción	Fecha y Hora	Acción
VWX-1122	Diana Carolina Vélez	Ingreso	5/1/2026, 7:45:56 p. m.	Finalizar
YZA-5566	Kevin Andrés Loor	Ingreso	5/1/2026, 7:45:35 p. m.	Finalizar

7.5.2. Registro de Visitantes

El guardia puede registrar visitantes externos de la siguiente manera:

1. Acceder a la opción Visitantes.
2. Ingresar los datos del visitante:
 - Nombres
 - Cédula
 - Placa del vehículo
 - Persona o facultad destino
 - Motivo de la visita
3. El sistema registra automáticamente la hora de ingreso.
4. Al finalizar la visita, se puede registrar la hora de salida mediante el botón correspondiente.

Panel de Control Guardia: **Guardia Juan** [Cerrar sesión](#)

Registro temporal de visitantes

Nombre del visitante: José Ramirez
Número de cédula: 1317059980
Placa del vehículo: KKK-9012
Persona / Facultad destino: Facultad de Derecho

Motivo de la visita: Entrega de documentos

Hora de ingreso: 08:51 p. m.

[Registrar ingreso](#) [Finalizar visita](#)

Visitantes activos

Nombre	Cédula	Placa	Destino	Motivo	Hora de ingreso	Acción
José Ramirez	1317059980	KKK-9012	Facultad de Derecho	Entrega de documentos	05:17 p. m.	Finalizar

7.6. Funcionalidades para el rol Administrador

7.6.1. Gestión de parqueaderos

El administrador puede registrar y visualizar los parqueaderos disponibles:

1. Acceder a Gestión de Parqueaderos.
2. Completar los campos:
 - Nombre del parqueadero
 - Ubicación
 - Capacidad total
 - Espacios disponibles
 - Estado (activo/inactivo)
3. Guardar la información para su visualización en la interfaz.

Esta funcionalidad es informativa y permite llevar un control general de los espacios disponibles.

Ilustración 39. Interfaz Gestión de Parqueaderos

ID	Nombre	Ubicación	Capacidad	Disponibles	Estado	Acciones
001	Parqueadero Facultad de Comunicación	Sector Norte - Puerta 2	40	15	Activo	Editar Eliminar
002	Parqueadero Ciencias de la Salud	Sector Central - Hospital Universitario	60	40	Activo	Editar Eliminar

7.6.2. Gestión de Usuarios Autorizados

Para registrar usuarios del campus, el administrador debe:

1. Acceder a Gestión de Usuarios.
2. Ingresar los datos solicitados:
 - Nombres completos
 - Cédula
 - Tipo de usuario
 - Facultad o departamento
 - Placa del vehículo
 - Tipo de vehículo
 - Correo institucional
3. Guardar la información.

Los usuarios registrados se muestran en una tabla y pueden ser buscados mediante un cuadro de búsqueda.

Panel de Control Administrador: Winter Povea [Cerrar sesión](#)

Registrar persona autorizada

Nombre completo: José Ramirez Cédula / ID ULEAM: 1317059980 Tipo de usuario: Estudiante Facultad / Departamento: TICS

Número de placa: KKK-9012 Tipo de vehículo: Auto Correo institucional: jose@uleam.edu.ec

[Registrar](#) [Cancelar](#)

Listado de personas autorizadas

Buscar por nombre, cédula, placa, facultad, tipo, vehículo...

ID	Nombre	Cédula	Tipo	Facultad / Departamento	Placa	Vehículo	Correo	Acciones
001	Luisa Torres	1312345678	Docente	Ciencias	ABC-	Automóvil	ltorres@uleam.edu.ec	Editar

7.6.3. Gestión de visitantes y descarga de historial

El administrador tiene acceso al registro de visitantes y puede:

- Descargar el historial en formato JSON para su respaldo o análisis.

7.7. Cierre de Sesión

Para cerrar la sesión:

1. Presionar la opción Cerrar sesión ubicada en la barra superior.
2. El sistema redirige automáticamente a la pantalla de inicio de sesión.

7.8. Consideraciones finales

El sistema fue diseñado con una interfaz simple y funcional, permitiendo un uso intuitivo sin necesidad de capacitación técnica avanzada. Todas las acciones realizadas quedan almacenadas localmente, garantizando la persistencia de la información durante la navegación.

8. Conclusión del Proyecto

El desarrollo del Sistema Web de Control de Parqueaderos ULEAM permitió aplicar de manera práctica los conocimientos adquiridos en la asignatura Aplicaciones Web I, integrando tecnologías modernas para la creación de una aplicación web funcional, organizada y orientada a la solución de una problemática real dentro del entorno universitario.

A través del uso del framework Vue.js y una arquitectura SPA, se logró implementar una interfaz dinámica, intuitiva y eficiente, capaz de gestionar el control de acceso vehicular, el registro de usuarios autorizados y visitantes, así como la administración informativa de los parqueaderos. La incorporación de herramientas como Pinia para la gestión de roles y Chart.js para la visualización de datos fortaleció la estructura y la experiencia de uso del sistema.

El uso de datos en formato JSON y LocalStorage permitió simular el funcionamiento de una base de datos, garantizando la persistencia de la información sin necesidad de un backend, lo cual resultó adecuado para el alcance académico del proyecto. Asimismo, la publicación del sistema en la plataforma Vercel evidenció su correcto funcionamiento en un entorno real de producción.

En conclusión, el proyecto cumplió satisfactoriamente con los objetivos planteados, demostrando la viabilidad de desarrollar aplicaciones web modernas utilizando buenas prácticas de programación y herramientas actuales. El sistema desarrollado sienta las bases para futuras mejoras, como la integración de una base de datos, autenticación avanzada y servicios backend, consolidándose como una propuesta escalable y adaptable a necesidades reales de la institución.

9. Bibliografía

- Vue.js. (2024). *Vue.js Documentation*. <https://vuejs.org>
- Vite. (2024). *Vite Documentation*. <https://vitejs.dev>
- Pinia. (2024). *Pinia – State Management for Vue*. <https://pinia.vuejs.org>
- Chart.js. (2024). *Chart.js Documentation*. <https://www.chartjs.org>
- Mozilla Developer Network (MDN). (2024). *Web Storage API (LocalStorage)*. <https://developer.mozilla.org>
- Vercel. (2024). *Vercel Documentation*. <https://vercel.com/docs>